UNIVERSIDAD POLITÉCNICA DE MADRID

E.T.S. DE INGENIERÍA DE SISTEMAS INFORMÁTICOS

PROYECTO FIN DE GRADO

GRADO EN INGENIERÍA DEL SOFTWARE

Sistema de préstamos descentralizado basado en la red Blockchain de Ethereum

Desarrollado por: Aquilino Pascual Sanz

Dirigido por: Alberto Díaz Álvarez y Guillermo Sánchez Gutierrez-Cabello

Madrid, 27 de octubre de 2026

Sistema de préstamos descentralizado basado en la red Blockchain de Ethereum
Desarrollado por: Aquilino Pascual Sanz
Dirigido por: Alberto Díaz Álvarez y Guillermo Sánchez Gutierrez-Cabello
Proyecto Fin de Grado, 27 de octubre de 2026

E.T.S. de Ingeniería de Sistemas Informáticos
Campus Sur UPM, Carretera de Valencia (A-3), km. 7
28031, Madrid, España

Si deseas citar este trabajo, la entrada completa en BibTeX es la siguiente:

```
@mastersthesis{citekey,
    title = {Sistema de préstamos descentralizado basado en la red Blockchain
de Ethereum},
    type = {Bachelor's Thesis},
    author = {Pascual-Sanz, A.},
    school = {E.T.S. de Ingeniería de Sistemas Informáticos},
    year = {2026},
    month = {10},
}
```

Esta obra está bajo una licencia Creative Commons «Atribución-NoComercial-CompartirIgual 4.0 Internacional». Obra derivada de https://github.com/blazaid/UPM-Report-Template.



Todo cambio respecto a la obra original es responsabilidad exclusiva del presente autor.

Agradecimientos

A mis compañeros de universidad,
A los que me ayudaron, y a los que ayudé,
A los que me enseñaron, y a los que enseñé.
Lo mejor de toda la universidad.

Resumen

Este proyecto de fin de grado nace con el objetivo de desarrollar un sistema financiero descentralizado que ofrezca transparencia, seguridad y automatización en la gestión de préstamos. La motivación principal para realizar este proyecto radica en la necesidad de eliminar intermediarios innecesarios, reducir costos operativos y garantizar la trazabilidad de cada operación mediante contratos inteligentes (Smart Contracts) en una red Blockchain. Este enfoque brinda mayor confianza a los usuarios y permite una gestión eficiente de los préstamos **sin depender de entidades centralizadas.**

Se ha desarrollado una solución basada en Blockchain, Smart Contracts y Web3 para descentralizar el proceso de préstamos, garantizando transparencia y eficiencia. Mediante contratos inteligentes, se automatiza la concesión, gestión y penalización por impago, reduciendo errores humanos. Además, se permite la integración con oráculos lo que podría validar datos externos como tasas de interés o solvencia . El sistema opera dentro de una dApp financiera, donde los préstamos se registran en la Blockchain y se gestionan de forma segura mediante un contrato principal que administra los fondos, asegurando un servicio innovador, verificable y eficiente.

Palabras clave: Blockchain, Smart Contract, DeFi, Web3, Préstamos automatizados.

Abstract

This bachelor thesis has been created with the aim of developing a decentralized financial system that offers transparency, security and automation in loan management. The main motivation for this project lies in the need to eliminate unnecessary intermediaries, reduce operating costs and ensure the traceability of each transaction through smart contracts (Smart Contracts) in a Blockchain network. This approach provides greater confidence to users and enables efficient loan management without relying on centralized entities.

A solution based on Blockchain, Smart Contracts and Web3 has been developed to decentralize the loan process, guaranteeing transparency and efficiency. Through smart contracts, the granting, management and penalties for non-payment are automated, reducing human errors. In addition, integration with oracles is enabled, which could validate external data such as interest rates or creditworthiness. The system operates within a financial dApp, where loans are registered in the Blockchain and securely managed through a master contract that manages the funds, ensuring an innovative, verifiable and efficient service.

Keywords: Blockchain, Smart Contract, DeFi, Web3, Automated loans.

Índice general

1 Introducción **1**

1.1 Objetivos 2

1.2 Motivación 3

1.3 Estructura de la memoria 4

2 Estado de la cuestión **6**

2.1 Protocolos de referencia en sistemas de préstamos descentralizados . . . 6

2.2 ¿Por qué Blockchain y por qué Ethereum? 8

2.3 Tecnologías para el desarrollo 15

3 Arquitectura y Desarrollo **19**

3.1 Solución propuesta 19

3.2 Limitaciones del proyecto 20

3.3 Análisis y diseño del software 22

3.4 Análisis Smart contracts implicados 32

3.5 Validación de resultados 44

4 Conclusiones **57**

4.1 Trabajo futuro 58

4.2 Impactos sociales y ambientales del desarrollo de este sistema a gran escala 59

A Código fuente de los Smart Contracts **63**

A.1 PRESTAMOS_FACTORY.sol 63

Índice de figuras

2.1 Diagrama simplificado del proceso de transacción en una blockchain. . . 11

2.2 Información a tiempo real del valor de criptomonedas, especialmente de *Solana* [14], *BNBChain* [15], *«Historia de Ethereum: fundador, lanzamiento y propiedad»* [17] las cuales sí permiten la integración de Smart Contracts. 13

3.1 Diagrama algoritmo de estados que asegura el control de los contratos del sistema. 20

3.2 Diagrama UML representando los actores del sistema relacionados con los casos de uso a los que tienen acceso. 23

3.3 Diagrama Secuencia que representa como la información vital del sistema queda registrada en la red aportando la inmutabilidad y confianza propia del proyecto 24

3.4 Diagrama entidad relación 32

3.5 Diagrama entidad relación que representa las relaciones y funciones entre los Smart contracts. 33

3.6 Ejemplo de login con Metmask 45

3.7 Información de contratos previa a la comprobación mensual en el recuadro amarillo y balance de la entidad financiera en el recuadro blanco. . . 46

3.8 Información de contratos posterior a la comprobación mensual en el recuadro amarillo y balance de la entidad financiera en el recuadro blanco. 47

3.9 Panel de control usuario. 48

3.11 Registro en Sepolia en el que se puede ver una transacción o comprobante de pago Etherscan 49

3.10 Información Etherscan del registro en la red SepoliaETH del SmartContract principal 49

4.1 Objetivos de desarrollo sostenible de la agenda europea para 2030 61

Índice de tablas

3.1 Casos de uso del sistema 25

3.2 Casos de uso para desarrollos posteriores 26

3.3 Casos de uso 1 26

3.4 Casos de uso 2 27

3.5 Casos de uso 3 27

3.6 Casos de uso 4 28

3.7 Casos de uso 5 28

3.8 Casos de uso 6 28

3.9 Casos de uso 7 29

3.10 Casos de uso 8 29

3.11 Casos de uso 9 29

3.12 Casos de uso 10 30

3.13 Casos de uso 11 30

3.14 Defínción de entidades del sistema 31

3.15 Descripción de atributos del Contrato Contract Factory 35

3.16 Descripción de funciones del Contrato Contract Factory 36

3.17 Descripción de atributos del Contrato Préstamo Contract 37

3.18 Descripción de funciones del Contrato Préstamo Contract 38

Índice de listados

3.1 Variable que contiene los prestamos producto 33

3.2 Variables contrato préstamo 34

3.3 Función para pagar mensualidad 38

3.4 struct TipoContrato 39

3.5 struct TipoContrato 40

3.6 Función que devuelve los contratos de un usuario 40

3.7 Recuperacion de informacion de contratos 42

3.8 Muestra de informacion de contratos 42

3.9 Despliegue de contrato en la red local 51

3.10 Parametrización de Prestamos cursando 52

A.1 Código fuente de PRESTAMOS_FACTORY.sol 63

1. Introducción

La tecnología blockchain ha transformado la forma en que entendemos las transacciones digitales, brindando un entorno seguro, transparente y, sobre todo, descentralizado, donde no es necesario depender de intermediarios financieros tradicionales y centralizados.

En un contexto económico global en el que algunos países enfrentan tasas de inflación elevadas, la adopción de monedas digitales como reserva de valor ha surgido como una alternativa atractiva para preservar el poder adquisitivo, intentando de algún modo evitar la erosión de valor que sufren las monedas locales.

Las criptomonedas ofrecen una opción descentralizada e internacional, proporcionando a los ciudadanos una alternativa que no está sujeta al control directo de los bancos centrales.

El aumento significativo de la inflación en algunas economías ha llevado a cuestionar el papel de los bancos centrales y sus políticas de control monetario, especialmente en lo que respecta a su influencia sobre la inflación y el valor de la moneda. Como respuesta a estas dinámicas, los criptoactivos han ganado popularidad en los últimos años como una forma de diversificación y preservación de valor en una economía global cada vez más conectada digitalmente.

La proliferación de proyectos DeFi refleja esta búsqueda de soluciones financieras alternativas que permiten a las personas interactuar directamente entre sí para obtener servicios financieros básicos, como préstamos y ahorros, sin intermediarios centralizados. Esta tendencia resalta el interés creciente por explorar formas de independencia económica, haciendo de las plataformas DeFi una propuesta relevante y de creciente adopción. A través de un sistema como el que se quiere proponer, que utiliza exclusivamente ETH y otros criptoactivos dentro de la red Ethereum, los usuarios podrían obtener préstamos respaldados por criptomonedas, sin la necesidad de instituciones financieras tradicionales y sin exponerse al impacto de decisiones de política monetaria centralizadas.

Este proyecto analiza si un sistema de préstamos descentralizado basado en Ethereum y

criptoactivos puede aportar mejoras respecto a los modelos tradicionales. A diferencia de estos, la descentralización permite reducir la intermediación, automatizar procesos mediante contratos inteligentes y ofrecer mayor transparencia en la gestión de los fondos.

Este trabajo pretende ofrecer una visión técnica y práctica de un sistema de préstamos DeFi, destacando sus oportunidades y desafíos, y evaluando su potencial para proporcionar una alternativa viable en el contexto financiero actual.

1.1. Objetivos

En esa sección se detallan los objetivos de este proyecto, estos se dividen en dos objetivos principales (Objetivo principal (OP)) y distintos objetivos secundarios (Objetivo secundario (OS)) los cuales surgen en el desarrollo de los objetivos principales.

1.1.1. Objetivos principales

- **OP 1**: Desarrollo de un sistema de préstamos automatizado sin una entidad centralizada controladora, de acceso libre y cumpliendo los requisitos básicos de un proceso de prestamo, como pueden ser, la seguridad, historial de cuotas, seguimiento, gestión de morosidad y personalización del usuario.
- **OP 2**: Desarrollar una interfaz unificada que permita a los usuarios, con o sin conocimientos técnicos, interactuar con el sistema de manera fácil e intuitiva. La interfaz garantizará que todas las funcionalidades sean accesibles desde un solo punto, y facilitando la experiencia del usuario.

1.1.2. Objetivos secundarios

- **OS 1**: Explorar la viabilidad de los sistemas de identificación en aplicaciones DeFi.
- **OS 2**: Crear una solución segura y eficiente. Almacenar únicamente la información estrictamente necesaria para el funcionamiento correcto de la aplicación evitando así la exposición de datos sensibles, y el procesamiento innecesario.

- **OS 3**: Optimizar eficiencia de ejecución de código en la red blockchain para reducir la cantidad de Gas necesario para el correcto funcionamiento del sistema.
- **OS 4**: Crear mecanismos de control de cumplimiento de contrato y penalización automatizado.
- **OS 5**: Explorar la flexibilidad y personalización de contratos inteligentes en productos financieros. Se investigarán maneras en las que el sistema sea capaz de ofrecer distintos productos financieros en función de la información que el sistema tenga del cliente como por ejemplo su situación financiera.
- **OS 6**: Optimizar el equilibrio entre la natuaraleza pública y transparente de una red Blockchain y la privacidad necesaria para los usuarios.
- **OS 7**: Analizar cómo se puede gestionar la liquidez en sistemas DeFi y cómo se puede coordinar la transferencia de fondos en un sistema de préstamos en una red Blockchain.

1.2. Motivación

Demostrar la viabilidad de un sistema de préstamos descentralizado mostrando cómo la tecnología Blockchain y los Smart Contract de Ethereum pueden ofrecer una alternativa sólida y segura frente a los sistemas financieros tradicionales, eliminando intermediarios y reduciendo costos y barreras de acceso.

Explorar y afrontar los desafíos técnicos específicos de desplegar un sistema DeFi en la red Ethereum.

Alcanzar la seguridad y eficiencia en los gastos del sistema, suponiendo esto una de las claves para conseguir la rentabilidad económica en el proyecto y pudiendose convertir en uno de los puntos débiles del sistema si no se realiza con detalle.

La motivación también radica en el valor social y económico que una solución como esta puede aportar a mundo en el que la inclusión financiera sigue siendo una meta distante para millones de personas. Los sistemas de préstamos descentralizados tienen el potencial de ampliar el acceso a servicios financieros básicos, independientemente del lugar de residencia o de la capacidad de acceso a instituciones tradicionales.

Enrique Dans, doctorado en sistemas de la información, habal en su blog que los proyectos DeFi responden a la creciente demanda de alternativas que no solo sean económicamente accesibles, sino también transparentes y seguras, proporcionando una herramienta que permita a los usuarios mantener una mayor autonomía sobre sus decisiones financieras en un entorno digital global. [1]

1.3. Estructura de la memoria

El código fuente de este proyecto se puede encontrar en el siguiente repositorio [2]. El código fuente con las pruebas unitarias de este proyecto se pueden encontrar en el siguiente repositorio [3]

La estructura del documento es la siguiente:

- **Introducción:**
  - Presentación del problema y contexto económico de los préstamos descentralizados.
  - Motivación y relevancia del proyecto.
  - Objetivos generales y específicos del sistema.
- **Estado de la cuestión**:
  - Introducción a Blockchain y su importancia en el sector financiero.
  - Explicación de Ethereum y su ecosistema.
  - Tecnologías clave (Web3, Solidity, Remix IDE, Node.js, Truffle, etc.).
- **Diseño de la Arquitectura:**
  - Solución propuesta: Cómo se estructura el sistema de préstamos descentralizado.
  - Suposiciones y limitaciones del proyecto.
  - Actores del sistema y su interacción (Usuarios, Propietarios, Contratos).
  - Casos de uso principales y pantallas del sistema.
  - Diagramas UML, diagramas de secuencia y diagramas entidad-relación.

- **Diseño detallado:**
  - Explicación a nivel técnico de cada entidad del sistema.
  - Smart Contracts implicados y su funcionamiento.
  - Explicación detallada de la lógica del algoritmo principal.
- **Conclusiones y trabajo futuro**
  - Evaluación de los logros alcanzados en el proyecto.
  - Posibles mejoras y nuevas funcionalidades a explorar.

2. Estado de la cuestión

2.1. Protocolos de referencia en sistemas de préstamos descentralizados

El ecosistema de las finanzas descentralizadas (DeFi) ha experimentado un crecimiento significativo en los últimos años, esto ha llevado a la aparación de distintos protocolos que ofrecen servicios de préstamos tal y como realiza este proyecto pero con distintas características específicas. El pequeño analisis que aparece a continuación no se va a centrar en la parte de negocio de estos protocolos sino en la parte tecnológica ya que cada uno de estos protolocos han sido creados por distintos equipos privadaos de desarrollo y no suponen un estándar en la industria de las finanzas descentralizadas. La integridad de estos protocolos se apoyan en las propiedades de las redes Blockchain, las cuales se hablan en este proyecto, y en la fuerza empresarial que han obtenido a través de la captación de usuarios y creación de comunidades.

Cabe destacar que estos protocolos se han desarrollado en distintas redes Blockchain y en el caso de ethereum se puede acceder a los Smart Contract desplegados en la red siendo público y accesible para cualquier usuario.

2.1.1. AAVE

El protocolo AAVE es el protocolo más parecido a la solución que ofrece este proyecto. La principal diferencia es que los usuarios finales no solo solicitan los préstamos si no que también pueden ofrecer sus fondos para que otros usuarios puedan solicitar préstamos.

El proceso consiste en una Pool de liquidez. Esta Pool es alimentada por los usuarios que ofrecen sus fondos y estos pueden ser pedidos como préstamo por otros usuarios. La definición del tipo de préstamo y sus intereses viene definida por la garantía que puede aportar la persona que solicita el préstamo, Dependiendo del valor ofrecido co-

mo garantía se le asignará un tipo de interés y un límite de préstamo.

Se puede comprobar su estado en la la red Ethereum en: *Estado del protocolo Aave V3 desplegado en la red* [4]

Para obtener mas información se puede consultar la documentación oficial del protocolo AAVE en: *Documentación del protocolo AAVE* [5]

2.1.2. Compound

Este protocolo es muy similar al anterior, la principal diferencia es que las garantias de devolución están previamente definidas según el tipo de préstamo que se solicita y no se permite la aportación de garantías externas.

Se puede comprobar su estado en la la red Ethereum en: *Estado del protocolo Compound III desplegado en la red* [6]

Para obtener mas información se puede consultar la documentación oficial del protocolo Compound en: *Documentación del protocolo Compound* [7]

2.1.3. Uniswap

Este protocolo es un intercambio descentralizado de criptomonedas, no es un protocolo orientado a préstamos aunque también se obtiene comisión a través de dejar prestados tus propios fondos en una Pool de liquidez. El funcionamiento consiste en aportar fondos a una Pool. En esta Pool se definen dos criptomonedas de tal forma que los usuarios aprovechan la liquidez de esos activos que se encuentran en la Pool para realizar intercambios de criptomonedas sin necesidad de un intermediario.

Por ejemplo, si un usuario quiere intercambiar ETH por Bitcoin, el protocolo Uniswap utiliza la liquidez de la Pool para realizar la transacción de manera automática y sin necesidad de conseguir un comprador que tenga exactamente Bitcoin y quiera exactamente ETH. En el contrato de intercambio se desglosa la comisión que obtiene la plataforma y el prestatario para la realización del intercambio.

Se puede comprobar su estado en la la red Ethereum en: *Estado del protocolo Uniswap V4 desplegado en la red* [8]

Para obtener mas información se puede consultar la documentación oficial del protocolo AAVE en: *Documentación del protocolo Uniswap* [9]

A continuación se definirán las tecnologías utilizadas en la solución ofrecida por este proyecto y también se definirán las claves por las que se han seleccionado dichas tecnología, desde la decisión inicial de utilizar una red blockchain hasta pequeñas decisiones de diseño como la selección del Framework para la UI

2.2. ¿Por qué Blockchain y por qué Ethereum?

2.2.1. Blockchain

Las redes Blockchain representan una revolución en el mundo digital ya que proporcionan una infraestructura segura, descentralizada y transparente para la transferencia y almacenamiento de datos en forma de transacciones.

Las Blockchain son bases de datos distribuidas, esto quiere decir que la información se guarda en distintos nodos en forma de bloques donde cada uno tiene un hash que vincula cada bloque con el anterior en una cadena ordenada. Gracias a este sistema de bloques en cadena registrados por un hash se hace imposible la modificación de la información guardada en la red sin que deje una huella digital, ya que al realizar este acto se modificaría el hash asociado al bloque afectado, haciendo visible para el resto de copias distribuidas en la red la modificación, seguramente fraudulenta.

Esta estructura en cadena proporciona las propiedades de inmutabilidad y resistencia a manipulaciones.

Estos sistemas permiten la creación, registro y transferencia de información o valor sin necesidad de una entidad central de control, de aquí su naturaleza descentralizada.

Este diseño y estructura presentan características innovadoras que posicionan a las redes Blockchain como una de las tecnologías más prometedoras, sobre todo en el sector financiero, aunque su aplicación se está extendiendo a diversas industrias tal y como se habla en el paper de la Universitat Politécnica de Catalunya [10]

Componentes en las transacciones Blockchain

Para comprender en profundidad el funcionamiento y la seguridad de una red blockchain, es fundamental analizar sus componentes clave, su estructura de datos, los mecanismos de consenso para la aportación de nueva información la red y las técnicas criptográficas que aseguran su integridad.

La siguiente información se ha obtenido a través del siguiente paper: «*INTRODUCCIÓN A LA TECNOLOGÍA BLOCKCHAIN: SU IMPACTO EN LAS CIENCIAS ECONÓMICAS S*» [11]

1. **Bloques**: La red Blockchain se estructura en bloques, que son conjuntos de información agrupada. Cada bloque contiene un conjunto de transacciones y un código de referencia al bloque anterior hash, formando una cadena continua. Los bloques también incluyen una marca de tiempo y un identificador único que garantiza su autenticidad.

2. **Hash criptográfico**: Cada bloque y cada transacción dentro de la blockchain está asegurada por un hash, que es un código único generado por los validadores. Este hash permite verificar de forma única el contenido del bloque y protege los datos de cualquier alteración. Si se modificara cualquier información en un bloque, el hash cambiaría, alertando al resto de la red de la alteración

3. **Nodos**: Son las computadoras que participan en la red. Cada nodo contiene una copia completa o parcial de la Blockchain y son responsables de verificar y propagar las nuevas transacciones y bloques. Los nodos se dividen en diferentes tipos:

   a) Nodos completos: Guardan la Blockchain completa y participan en la verificación de bloques.

   b) Nodos ligeros: Descargan partes de la cadena y confían en nodos completos para la verificación.

   c) Nodos mineros: Son los encargados de replicar la blockchain, aceptando los bloques generados por los validadores y ejecutando las transacciones incluidas en los mismos. Estos nodos son los que inyectan transacciones en la red, a partir de fuentes externas al blockchain. Estos nodos compiten para agregar nuevos bloques a la cadena realizando el proceso de validación de transacciones y siendo incentivados por su labor

4. **Proceso de validación de bloque**: Los nodos mineros pueden validar los nuevos bloques a través de distintos procesos, los que incumben a este proyecto son:

a) ***Proof of work*** **(Proof of Work, Prueba de trabajo (PoW))**: Forma de validación en la que los nodos mineros compiten por resolver un problema matemático que implica encontrar un hash que cumpla ciertos requisitos establecidos por el protocolo. El primer nodo en obtener este hash es el que se encarga de incorporar el nuevo bloque a la red y obener la recompensa. Este proceso es dificil de resolver pero facil de verificar para otros nodos.

b) ***Proof of Stake*** **(Proof of stake, Prueba de participación (PoS))**: Los participantes de la red depositan una cantidad de criptomonedas en fomra de «participación». Estos participantes se llaman validadores. Los validadores son seleccionados de forma pseudo-aleatoria para proponer y validar nuevos bloques, con probabilidades generalmente proporcionales a la cantidad de criptomonedas que tienen participando. Si el validador elegido obiene el hash de forma correcta recibe la recompensa, y si actúa de manera malicionsa puede perder su participación.

5. **Libro mayor distribuido**: La Blockchain en conjunto actúa como un libro de contabilidad digital distribuido. Este libro mayor es inmutable, lo que significa que una ç vez registrada una transacción, no se puede modificar. Todos los nodos tienen acceso a la misma información, y las actualizaciones se sincronizan de forma continua y automática.

Proceso de transacción

Cada transacción en una blockchain sigue un proceso de verificación y registro antes de formar parte de un bloque:

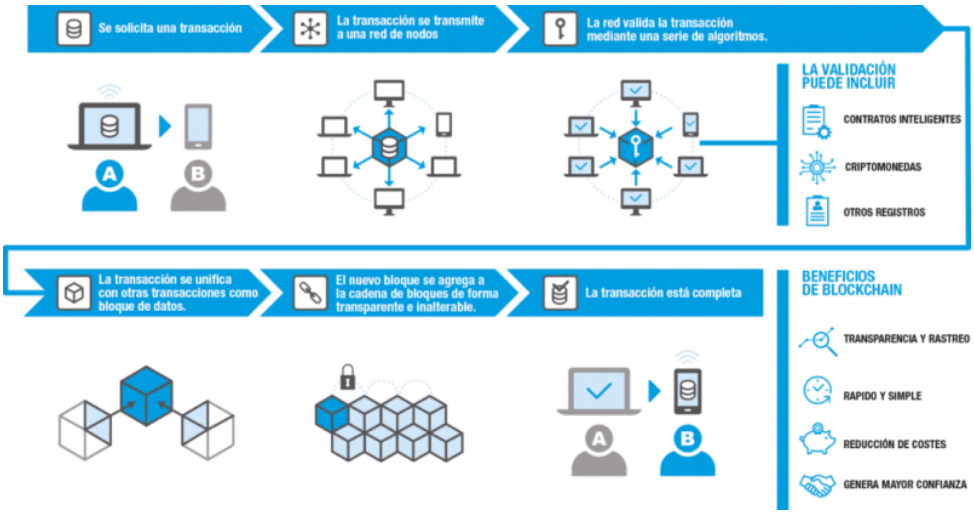
1. **Creación de la transacción**: El usuario (o nodo emisor) crea una transacción con información como las direcciones del emisor y receptor, el monto de la transferencia y la firma digital generada con la clave privada del emisor.
2. **Verificación de la transacción**: Los nodos validadores comprueban que la transacción es válida mediente PoW o PoS.
3. **Difusión a la red**: La transacción es difundida a la red para su verificación. Los nodos recopilan las transacciones válidas y las agrupan en bloques para su posterior validación.
4. **Agregación a la blockchain**: El bloque verificado se agrega a la cadena. Este proceso implica registrar la transacción inicial y todas las demas del bloque y vincular

el nuevo bloque al bloque anterior mediante un hash. Esto asegura que la cadena sea inmutable y cronológica.

5. **Confirmación de la transacción**: Una vez que un bloque ha sido agregado, la transacción es confirmada. Las confirmaciones se refuerzan con cada bloque añadido posteriormente, haciendo la transacción prácticamente irreversible.

A través de la figura 2.1 se representa un diagrama simplificado del proceso de transacción.

Figura 2.1. Diagrama simplificado del proceso de transacción en una blockchain. Fuente: Neocheck [12]



Seguridad en Blockchain

La seguridad de la blockchain depende de varias capas de tecnología criptográfica y del diseño descentralizado de la red. Los elementos clave de su seguridad incluyen:

1. **Inmutabilidad**: Una vez que se agrega un bloque, es extremadamente difícil modificar los datos sin alterar el hash que contiene sin afectar a todos los bloques posteriores. Cualquier intento de modificación es rápidamente detectado por los nodos validadores.

2. **Hashing criptográfico**: El uso de algoritmos de hashing (como SHA-256 en Bitcoin) convierte los datos de las transacciones en códigos únicos de longitud fija. Cualquier cambio en los datos originales generaría un hash completamente diferente, alertando a la red de la alteración. Este proceso es irreversible, lo que significa que el hash no puede descifrarse para revelar los datos originales.

3. **Distribución y descentralización**: Al distribuir el control de la red entre miles de nodos (en el caso de las redes más importantes), Blockchain elimina los puntos de fallo únicos. Esto significa que, incluso si un nodo es comprometido o hackeado, el resto de la red sigue siendo seguro, de hecho dependiendo de los criterios de cada red, realmente mientras ente el 51 y 70 % de nodos siga sin alterarse la red sigue siendo segura.

4. **Criptografía de clave pública y privada**: Cada usuario de la red tiene una clave pública y una clave privada. La clave pública funciona como una dirección visible para recibir activos e identificarse en la red ante otros usuarios, mientras que la clave privada permite firmar transacciones y verificar la identidad del usuario, asegurando la autenticidad de la actividad en la red. Solo el propietario de la clave privada puede autorizar transacciones en nombre de su dirección pública.

Con todas estas características se puede afirmar que una red Blockchain es una herramienta segura para la realización de transacciones monetarias y de reserva de valor. Ahora bien, ¿cómo se puede crear un sistema automatizado?. En redes como Bitcoin el único tipo de operación que se puede realizar es el de transacción. Este proyecto se apoya en una red con capacidad de ejecutar scripts o como son llamados dentro de la red de Ethereum: Smart Contracts.

2.2.2. Ethereum

Historia

La información presentada a continuación sobre el origen de Ethereum se ha obtenido de su página web oficial [13].

Ethereum es una plataforma de Blockchain propuesta en 2013 por Vitalik Buterin a sus 19 años de edad y fue lanzada oficialmente en 2015. Es una plataforma descentralizada gobernada por su comunidad y con el apoyo de la fundación Ethereum, sin ánimo de lucro.

Buterin mostró un talento matemático excepcional a edad temprana y comenzó a escribir artibulos sobre Bitcoin, llegando a cofundar «Bitcoin Magazine 2012».

En 2014. publicó el «Documento técnico de Ethereum», en el que habla de la nueva

plataforma que iría más allá del Bitcoin creando una cadena de bloques que podría hacer más que solo pagos.

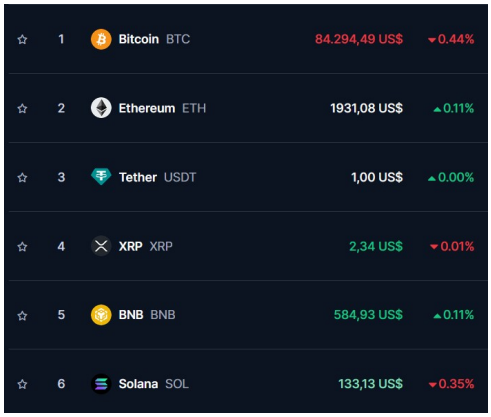
> "Ethereum amplía el enfoque de Bitcoin, básicamente diciendo: bueno, en lugar de tener reglas diseñadas para respaldar una sola aplicación, vamos a crear algo de propósito más general donde las personas puedan simplemente crear sus propias aplicaciones y las reglas para cualquier aplicación que hayan creado puedan ejecutarse e implementarse en la plataforma Ethereum." Ethereum.org [13]

2.2.3. Ethereum para el proyecto

El poderoso desarrollo de Vitalik es el motivo principal por el que se ha escogido la red de Ethereum: la capacidad de desplegar Smart Contracts.

Aunque existen otras redes Blockchain con capacidad de despliegue de Smart Contracts, como *Solana* [14] y *BNBChain* [15], al tratarse de un proyecto basado en la economía DeFi se ha seleccionado Ethereum debido a que su moneda es la que tiene mayor valor de mercado. Este se puede visualizar en la figura 2.2 a fecha de 14-03-2024 y a tiempo real en la web de *Coin Market Cap* [16]

Figura 2.2. Información a tiempo real del valor de criptomonedas, especialmente de *Solana* [14], *BNBChain* [15], *«Historia de Ethereum: fundador, lanzamiento y propiedad»* [17] las cuales sí permiten la integración de Smart Contracts.



Las características de Ethereum en las cuales se apoya este proyecto son:

1. **Contratos Inteligentes y Máquina Virtual de Ethereum (Ethereum Virtual Machine, Máquina Virtual de Ethereum (EVM))** Como se ha comentado antes, esta es la característica más importante de ethereum. Los Smart Contract son programas autoejecutables con condiciones predefinidas, que permiten automatizar una amplia variedad de procesos y acuerdos entre partes sin necesidad de intermediarios. La Máquina Virtual de Ethereum (EVM) es la infraestructura que hace posible la ejecución de estos Smart Contract. Esta consiste en un entorno virtual que permite que el código de los Smart Contract se ejecute por igual y de manera segura en todos los nodos de la red Ethereum, lo que garantiza consistencia e inmutabilidad. La EVM es compatible con varios lenguajes de programación, siendo Solidity el más utilizado para desarrollar contratos inteligentes en la red y el que se ha utilizado en este proyecto.

2. **Plataforma para dApps** A partir del desarrollo de los Smart Contract, Ethereum fue la primera Blockchain en introducir el concepto de aplicaciones descentralizadas (dApps). Las dApps son aplicaciones que funcionan en redes descentralizadas, permitiendo que los usuarios interactúen de manera directa y sin intermediarios en diferentes servicios, desde finanzas hasta juegos y redes sociales. En este caso, EVM actúa como una «App Store descentrelizada»

3. **Criptoeconomía y Modelo de Tokenización** La existencia de la criptomoneda ETH en Ethereum es clave para el desarrollo de un sistema DeFi, pudiendo ser utilizada para realizar transacciones y pagar comisiones obligatorias.

   Además, Ethereum introdujo el concepto de «tokenización», esto permite representar activos o derechos a través de Token. Para facilitar el uso y transferencia de estos, se han creado distintos estandares como por ejemplo:

   a) ERC-20 [18]: tokens fungibles, cuyas unidades son intercambiables entre sí, como puede ser una criptomenda.

   b) ERC-72 [19] tokens no fungibles, que permiten distinguir cada token individualmente, pudiendose usar para representar activos en la vida real o distintos derechos de usuario como una votación.

   c) ERC-1155 [20]: permite gestionar distintos tipos de tokens en una misma operación, tanto fungibles como no fungibles, en un mismo contrato.

4. **Identidad** Ethereum es una red descentralizada, no utiliza identificaciones centralizadas como puede ser la centralización estatal del Documento Naciona de Identidad (DNI). Aunque no es una característica exclusiva de Ethereum, el sistema de identificación de esta red consiste en el típico sistema asimétrico de clave

pública y privada de forma que para identificar a cada usuario se utiliza su clave pública y para que este pueda ejecutar transacciones en su nombre utiliza su clave única y privada. Sin embargo, esto también significa que no hay una forma nativa de verificar identidades del mundo real.

5. **Oráculos** son servicios externos que alimentan la blockchain con datos de fuentes externas, permitiendo que los contratos inteligentes interactúen con el mundo real. Por ejemplo, en aplicaciones DeFi relacionadas con bienes raíces, un oráculo podría verificar que una propiedad física corresponde a un token en Ethereum, permitiendo así transacciones con respaldo en activos físicos.

Se puede concluir que ethereum es un buen medio para desarrollar la intencionalidad del proyecto gracias a su EVM y la posiblidad de definir Smart Contract en ella, utilizando como moneda de cambio ETH e incluso añadir avales a los contratos y préstamos utilizando los NFT e identificándolos en el mundo físico a través de «oráculos».

2.3. Tecnologías para el desarrollo

2.3.1. Herramienta de identificación: Metamask

La aplicación web necesita conectarse con la cuenta y dirección Ethereum del usuario para solicitar la firma de las operaciones de préstamo.

La primera opción podría ser desarrollar a través de herramientas de desarrollo Web3 una capa de identificación en la red, aunque se ha decidido delegar esta tarea a proyectos ya existentes y consolidados ya que es un punto crítico de seguridad del sistema y de esta manera se evita almacenar claves privadas de usuario.

Existen varias alternativas externas para identifición en Blockchains en el sistema como Coinbase [21] y RabbyWallet [22] aunque simplemente se ha optado por Metamask por su popularidad.

Estas herramientas de identificación proporcionan una API para comunicar la dApp con la Wallet pudiendo automatizar a traves de funciones Web3 las peticiones de autorización al usuario que ha entrado en el sistema.

En el caso de Metamask, esta actúa como extensión del navegador y aparece como un Pop-up cada vez que la dApp requiere una autorización, también sirve para hacer transferencias directas entre cuentas, y por su puesto sirve de Wallet

2.3.2. Herramienta multiusos para despliegue en EVM Truffle

Para el desarrollo de una dApp en Ethereum Virtual Machine (EVM) es necesario realizar distintos procesos de compilación y despliegue para probar el funcioniento del sistema antes de desplegar en la Blockchain

Truffle [23] es un conjunto de herramientas para el desarrollo de aplicaciones en EVM. Principalmente puede compilar Smart Contracts y reducir el código a la información necesaria la EVM

Truffle también tiene una herramienta muy poderosa: Ganache [24], esta es capaz de ejecutar una EVM en un entorno local. Perfecto para el desarrollo aunque tiende a fallar.

Red de pruebas Sepholia-ETH Aunque de manera muy técnica sea posible crear Blockchain de manera local, debido a la importancia de probar todo desarrollo antes de desplegarlo en la red inmutable, existen alternativas de prueba de la mayoría de las redes Blockchain públicas, estas actúan exactamente igual que la red original pero si utilizar criptomonedas reales. Sepolia [25] es la red encargada de simular la red Ethereum, su moneda se denomina «SepoliaETH» y se puede obtener de manera gratuita en dsitintas plataformas.

2.3.3. Explorador de bloques Etherscan

La información guardada en la red está en distintas capas y de forma no legible para el humano. Etherscan [26] es un explorador de bloques de Blockchain para Ethereum que permite visualizar transacciones, Smart Contract, balances de Wallets y más información en tiempo real. Es una herramienta clave para analizar y verificar actividades en la red Ethereum.

2.3.4. Remix IDE

Se ha de tener en cuenta que en el desarrollo de esta dApp aún no estaban tan extendidas las herramienta de desarrollo centralizado a través de inteligencia articifial, por lo que se encontró una alternativa muy buena para el desarrollo de Smart Contracts con Solidity.

Remix Project [27] es el conjunto de herramientas más exhausitivo que puede usarse para toda la experiencia de desarrollo de Smart Contracts. Ofrece todo tipo de herramientas para la depuración de código, despliegue en redes tanto públicas como privadas, compilación de Smart Contract, etc…

2.3.5. Test unitarios: Mocha Framework

Marco de pruebas en Javascript diseñado para ejecutar pruebas unitarias y de integración en aplicaciones Node.js. Ofrece la capacidad de comprobar reglas comunes a los Smart Contracts como la comprobación de permisos de usuario, ingreso y retirada de fondos.

Tiene funciones para esperar operaciones asíncronas, muy importante a la hora de esperar acciones provenientes de la EVM, a demás de las funciones comunes a cualquier otro entorno de trabajo para tests.

2.3.6. Node.js

Node.js [28] es un entorno de ejecución Javascript de código abierto y gratuito que facilita la incorporación de paquetes a un proyecto. Muy utilizado para instalar y utilizar librerías Web3.

2.3.7. Web3

Se le llama web3 a la próxima generación web (actual web2). Entre otras cosas buscará la descentralización y sobretodo el uso de dApp. A su vez web3 es un Framework de Javascript que permite acciones como conectarte a una red bockchain, gestionar cuentas,

firmar transacciones, consultar saldos de las cuentas, interactuar con Smart Contracts. Este framework abstrae la complejidad de las interacciones con la Blockchain, simplificando el desarrollo de dApps.

2.3.8. Solidity

Solidity [29] es un lenguaje de programación diseñado para escribir Smart Contracts que se ejecutan en la EVM de Ethereum. Es similar a JavaScript y C++, con características específicas para gestionar transacciones, almacenamiento en blockchain y lógica descentralizada.

Aunque existen alternativas para el desarrollo de Smart Contract, solidity tiene más comunidad enfocada directamente al desarrollo de Smart Contract, lo cual es clave en un entorno «pre-Inteligencia Artificial»

2.3.9. Angular

Angular es un framework de código abierto basado en TypeScript. Sus herramientas hacen más sencillo las tareas comunes al desarrollo web como la gestión de rutas de dirección, organización de código a través de elementos llamados «componentes», recogida de datos de usuario, etc..

2.3.10. Ionic

Ionic es un framework de código abierto orientado en el desarrollo de los componentes visuales en una aplicación web. En combinación con *Angular* se convierte en una herramienta que facilita mucho el desarrollo web.

3. Arquitectura y Desarrollo

3.1. Solución propuesta

Este proyecto propone una dApp que gestiona préstamos de ETH dentro de la red Ethereum a través de Smart Contract desplegados en dicha red.

Como se ha explicado en la comparación con otros protocolos DeFi, la diferenciación de este sistema es que solo una entidad, possedora del proyecto, está autorizada a ofrecer los prestamos, configurar los tipos de préstamos disponibles y gestiona los fondos.

El sistema principalmente se basa en en el diseño de los Smart Contracts que junto a las propiedades de inmutabilidad y veracidad de una Blockchain deja escrita en la red la información necesaria para llevar un libro contable de un sistema de préstamos de forma casi automatizada.

El usario de la dApp interactúa con el sistema a través de una interfaz web en la que conectando su cartera con Metamask puede gestionar sus propios préstamos sin la necesidad de interactuar directamente con la organización del sistema.

El seguimiento de control de pagos de los contratos funciona como una máquina de estados automatizada que es accionada como un semáforo con ciclos mensuales.

En la figura 3.1 se pueden ver los tres tipos de estados en los que puede llegar un préstamo al chequeo mensual de pagos (estado 0), como actúa la dApp dependiendo de su estado durante el chequeo (estado 1), y como se queda el préstamo pasado el chequeo esperando al próximo (estado 0).

Figura 3.1. Diagrama algoritmo de estados que asegura el control de los contratos del sistema.

ESTADO 0	ESTADO 1	ESTADO 0
	 **CHEQUEO DE MENSUALIDAD**	
PLAZO ABIERTO	PLAZO ABIERTO (PENALIZACIÓN)	PLAZO ABIERTO
MES PAGADO	MES PAGADO (CONTINUA CON NORMALIDAD)	PLAZO ABIERTO
FINALIZADO	FINALIZADO (SE TANSFIEREN FONDOS A CONTRACT FACTORY Y SE BORRA DE MAPPING DE PRÉSTAMOS ACTIVOS)	FINALIZADO (ESPERANDO BORRADO)

3.2. Limitaciones del proyecto

3.2.1. Limitaciones legales

Durante este proceso de desarrollo, en diciembre de 2024, la Unión Europea aprueba el Reglamento Europeo de Criptoactivos [32], el objetivo de este reglamento es establecer normas para las entidades emisoras y proveedoras de servicios relacionados con la criptoactividad.

En el nuevo reglamento se tratan distintas cuestiones entre las que se encuentran:

1. Requisitos de transparencia absoluta a las plataformas de negociación de criptoactivos.
2. Necesidad de autorización europea para lea emisión y provisionamiento de criptoactivos.
3. Protección del valor monetario en euros de los clientes.
4. Creación de medidas de control de las operaciones con información privilegiada y manipulación de mercado.

En esta normativa se especifican acciones obligatorias para entidades financieras que pueden adoptar este proyecto. Dentro de estas acciones se definen elementos comunes a cualquier empresa como definirse como persona jurídica o actuar de forma honesta, justa y profesional.

Las acciones que pueden llegar limitar el proyecto es la obligatoriedad de ofrecer la capacidad de convertir en Euros los criptoactivos proporcionados por este proyecto. Específicamente se obliga a mantener:

1. Un fondo propio de 350.000E
2. El 2 % del importe medio de los activos de reserva
3. Una cuarta parte de los gastos generales fijos del año anterior
4. Establecimiento de planes de recuperación y reembolso en caso de no cumplir las obligaciones de la normativa

Estas acciones obligan a cualquier entidad que quiera hacer uso de este proyecto a aprovisionar las cantidades definidas en la normativa.

3.2.2. Limitaciones técnicas

El sistema desarrollado presenta una serie de limitaciones técnicas derivadas tanto del alcance definido para el proyecto como de las características propias de una solución basada en Smart Contract. Las principales limitaciones identificadas son las siguientes:

1. **Pagos anticipados:** el sistema no contempla actualmente la realización de pagos anticipados sobre los préstamos contratados.
2. **Cancelación anticipada:** no se contempla la posibilidad de cancelar un préstamo antes de que finalice el periodo establecido inicialmente en el momento de su contratación.
3. **Ausencia de garantías o colateral:** el sistema no incorpora mecanismos de colateralización que permitan garantizar el pago del préstamo en caso de impago.
4. **Uso de un único activo:** el prototipo utiliza exclusivamente ETH como activo para la concesión y devolución de los préstamos. No se contempla actualmente el uso de otros tokens o activos digitales como unidad de préstamo.
5. **Escalabilidad y consumo de Gas:** el incremento del número de préstamos activos implica un mayor número de operaciones que deben ser procesadas y registradas en la Blockchain. En consecuencia, el coste acumulado de Gas puede aumentar a medida que crece el volumen de préstamos gestionados. Esta situación no tiene por qué invalidar el proyecto pero sí ejecutar un desarrollo enfocado en la escalabilidad.

Estas limitaciones deben tenerse en cuenta al valorar una posible aplicación del sistema en un entorno real a la vez que se plantean como líneas de desarrollo futuro.

3.3. Análisis y diseño del software

En esta sección se muestran diagrmas y tablas que definen la estructura y flujo del sistema.

3.3.1. Actores del sistema

Los actores que pueden participan en el sistema son:

- **Usuario prestatario (User)** Usuario que accede al servicio de préstamos.
- **Entidad prestamista (Owner)** Entidad que ofrece los distintos tipos de préstamos.

Diagrama UML

En la figura r3.2 hay representado un Diagrama-UML que relaciona los actores del sistema con los «Casos de Uso» que pueden realizar.

Figura 3.2. Diagrama UML representando los actores del sistema relacionados con los casos de uso a los que tienen acceso.

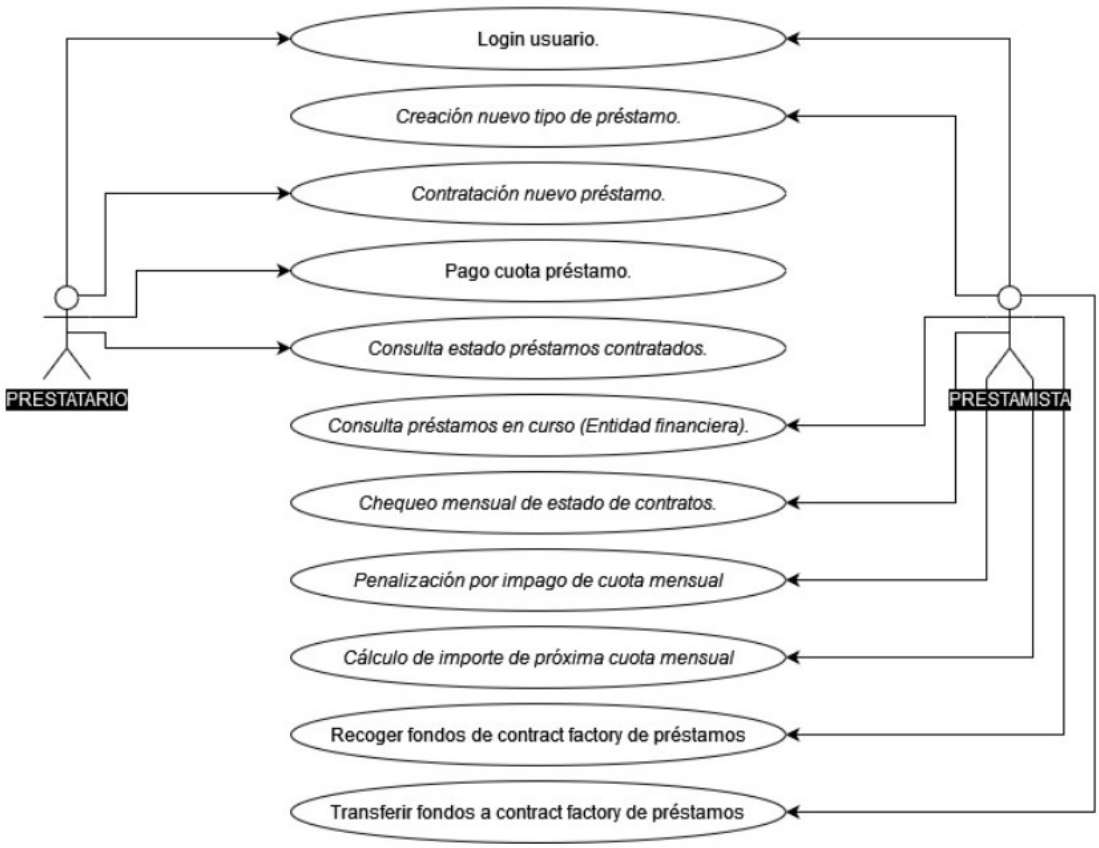
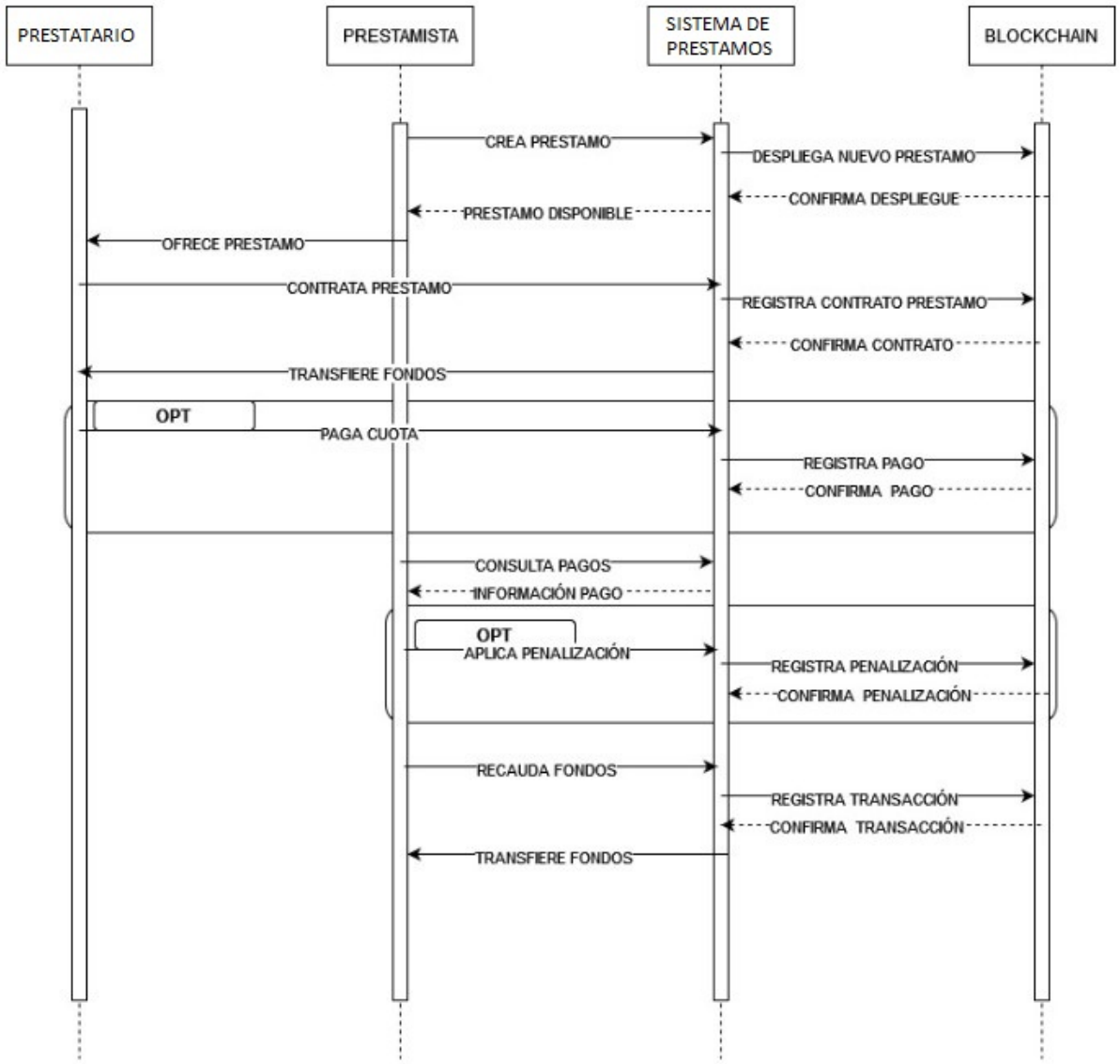


Diagrama de secuencia UML

Para mostrar como interactúan los actores con el sistema en una lína de tiempo y conocer el flujo de información que sucede en el sistema se ha definido el siguiente Diagrama de Se cuencia-UML en la figura 3.3

En este caso, el diagrama de secuencia muestra como todo registro en el sistema siempre es grabado en la Blockchain, siendo un flujo clave en este proyecto para poder dar las propiedades de veracidad e inmutabilidad a la información de los préstamos.

Figura 3.3. Diagrama Secuencia que representa como la información vital del sistema queda registrada en la red aportando la inmutabilidad y confianza propia del proyecto



3.3.2. Casos de uso

En el marco del desarrollo del presente proyecto, se ha priorizado la definición de un conjunto de casos de uso que representan los escenarios esenciales para cumplir con los objetivos fundamentales del sistema ya que este proyecto debe de ser 'finito' y no 'infinito'.

La selección de los casos de uso no pretende abarcar una totalidad de posibles funcionalidades del sistema sino un conjunto operativo sólido y funcional, comentando también un posible desarrollo futuro.

Estos casos de uso se definen en las tablas 3.1 y 3.2

Pantallas del sistema

Para un mayor entendimiento de los casos de uso se realiza una declaración de las posibles pantallas que participan en el sistema son:

- **Pantalla principal común** Pantalla de incio comun a todos los usuarios, principalmente para identificarse en el sistema.
- **Pantalla principal de prestamista.**
- **Pantalla principal de prestatario.**

Casos de uso

Tabla 3.1. Casos de uso del sistema

Código	Descripción	Prioridad	Dependencia
CU1	Login usuario	10	
CU2	Creación nuevo tipo de préstamo	9	
CU3	Contratación nuevo préstamo	8	CU2
CU4	Consulta estado préstamos contratados	7	CU3
CU5	Pago cuota préstamo	6	CU3
CU6	Consulta préstamos en curso (Entidad financiera)	5	CU3
CU7	Chequeo mensual de estado de contratos	4	
CU8	Penalización por impago de cuota mensual	3	CU7
CU9	Cálculo de importe de próxima cuota mensual	2	CU3
CU10	Recoger fondos del prestatario	1	
CU11	Transferir fondos al prestamista	1	

Casos de uso para posteriores desarrollos

Tabla 3.2. Casos de uso para desarrollos posteriores

Código	Descripción
CU12	Garantía de devolución en caso de impago
CU13	Consulta del valor del Euribor
CU14	Validación de solvencia económica de usuarios

Casos de uso detallados

En esta sección se describen los casos de uso implementados y más adelante, en la sección de trabajo futuro se comentarán los casos de uso futuro.

Tabla 3.3. Casos de uso 1

CU1	Log-In usuario.
Descripción	Los usuarios se identificarán en el sistema a través de Metamask
Pantalla	Pantalla inicial. (Se redirecciona a esta página si no se ha realizado el Log-In)
Rol	Cualquier usuario
Flujo	- El usuario, al acceder a la página web, recibirá una notificación por parte de Metamask para poder acceder al servicio - La extensión de Metamask permite la autenticación del usuario y la página se redirige a la respectiva página correspondiente segun su rol de usuario.

Tabla 3.4. Casos de uso 2

CU2	Creación nuevo tipo de préstamo.
Descripción	La entidad prestamista puede crear nuevos productos tipo "préstamo"
Pantalla	Pantalla principal de la entidad prestamista.
Rol	Owner
Flujo	- El usuario owner tiene un formulario en su página principal para rellenar la información necesaria para la creación de un nuevo tipo de préstamo. - Este contrato se le mostrará al usuario prestatario en su página principal en forma de producto para contratar.

Tabla 3.5. Casos de uso 3

CU3	Contratación nuevo préstamo.
Descripción	Los usuarios prestatarios pueden seleccionar el préstamo que desean contratar
Pantalla	Pantalla principal de usuarios prestatarios.
Rol	User
Flujo	- En la página principal el usuario podrá visualizar los posibles préstamos que puede contratar. - El usuario contrata un nuevo préstamo pinchando en el botón de "Contratar". Tan pronto como se procese la transacción, la dApp llamará al Smart Contract desplegado en la red el cual genera otro Smart Contract que refleja el estado del préstamo en cada momento. Al mismo tiempo, las unidades de ETH solicitadas en el préstamo serán transferidas automáticamente desde los fondos del Smart Contract que conserva el balance del sistema a la dirección del usuario.

Tabla 3.6. Casos de uso 4

CU4	Consulta estado préstamos contratados
Descripción	Los usuarios del servicio podrán consultar el estado de sus préstamos contratados visualizándolos en su página principal
Pantalla	Pantalla principal usuario
Rol	User
Flujo	En el momento que el usuario hace log in ya puede visualizar los contratos que tiene contratados

Tabla 3.7. Casos de uso 5

CU5	Pago cuota préstamo.
Descripción	Los usuarios del servicio de préstamo realizan el pago de sus mensualidades a través de la dApp del proyecto
Pantalla	Pantalla principal de usuarios prestatarios
Rol	User
Flujo	-En la página principal del usuario aparecen los distintos prestamos que tiene contratados y su respectivos estados, junto a esto, existe un botón para cada contrato que transfiere la mensualidad calculada previamente desde la cuenta del usuario a la dirección del Smart Contract de préstamo.

Tabla 3.8. Casos de uso 6

CU6	Consulta préstamos en curso (Entidad financiera).
Descripción	La entidad financiera puede consultar el estado de cada préstamo en curso
Pantalla	Pantalla principal de la entidad financiera
Rol	Owner
Flujo	-En la página principal del propietario aparecen los contratos en curso

Tabla 3.9. Casos de uso 7

CU7	Chequeo mensual de estado de contratos.
Descripción	La entidad financiera ejecuta la orden de comprobar que mensualidades no han sido pagadas ese mes.
Pantalla	Pantalla principal de entidad financiera
Rol	Owner
Flujo	Desde la página principal del propietario se tiene acceso al botón que lanza esa orden siempre y cuando hayan pasado 30 días desde la última orden.

Tabla 3.10. Casos de uso 8

CU8	Penalización por impago de cuota mensual
Descripción	Los contratos en estado de impago generan una penalización de deuda, esta penalización solo puede suceder una vez al mes.
Pantalla	Pantalla principal de entidad financiera
Rol	Owner
Flujo	Cuando el propietario genera la orden de chequeo mensual los contratos que tengan estado de «impago» aumentaran su valor de deuda

Tabla 3.11. Casos de uso 9

CU9	Cálculo de importe de próxima cuota mensual
Descripción	El importe de la próxima cuota mensual es un valor dinámico sujeto a penalizaciones
Pantalla	Pantalla principal usuario
Rol	User
Flujo	- Tras el chequeo mensual la cuota del próximo pago se podrá ver modificada en caso de penalización - En la información de los préstamos contratados aparecerá la cantidad de la cuota del mes en curso

Tabla 3.12. Casos de uso 10

CU10	Recoger fondos del contrato de préstamos
Descripción	La entidad bancaria propietaria del sistema de prestámos puede disponer de los fondos almacenados en el sistmea cuando desee
Pantalla	Pantalla principal de Owner
Rol	Owner
Flujo	La entidad bancaria tendrá una casilla en su página principal para escribir y recibir el importe desde la dirección del Smart Contract con el balance a su dirección de la red

Tabla 3.13. Casos de uso 11

CU11	Transferir fondos al sistema de préstamos
Descripción	La entidad bancaria puede transferir cualquier cantidad de ETH al sistema de prestamos para mantener un saldo positivo y poder realizar estos préstamos
Pantalla	Pantalla principal Owner
Rol	Owner
Flujo	La entidad tendrá una casilla en su página principal para escribir y transferir el importe deseado a la dirección del sistema de préstamos.

3.3.3. Entidades del sistema

A continuación se definen las entidades cuyas relaciones compondrán y habilitarán las funciones del sistema propuesto

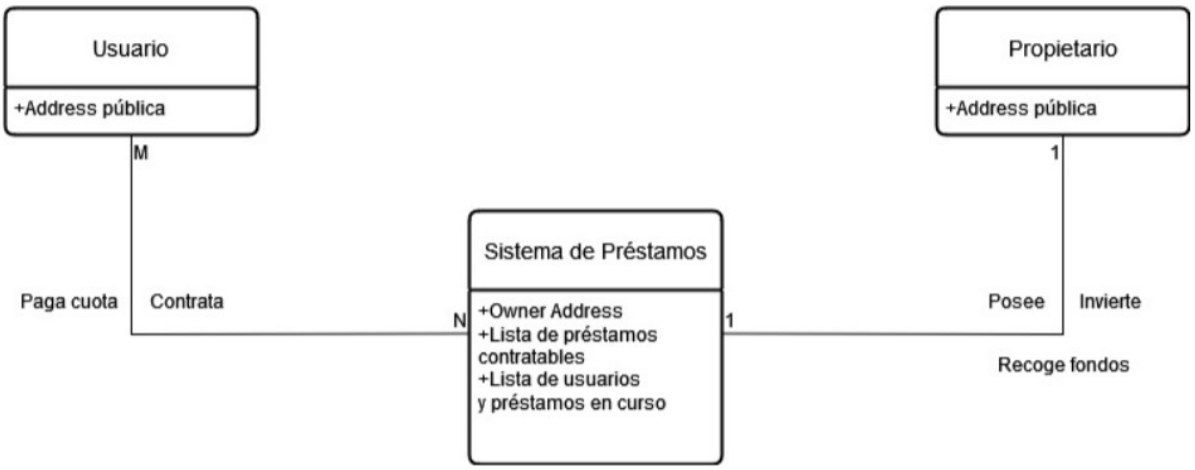
Tabla 3.14. Defincíon de entidades del sistema

1	Usuario.
Descripción	Usuario que accede al servicio de préstamos.
Atributos	Dirección pública de la red.
Acciones	- Pedir préstamo - Pagar préstamo
2	Propietario.
Descripción	Propietario del sistema, único capaz de recolectar los fondos del sistema. Normalmente una entidad financiera
Atributos	Dirección pública de la red.
Acciones	- Transferir unidades al sistema - Recoger unidades del sistema
3	Sistema de préstamos
Descripción	Conjunto de Smart Contract que definen los elementos del sistema y cuya ejecución valida las transacciones del sistema
Atributos	- Dirección de propietario - Lista de préstamos contratables - Lista de usuarios junto con sus préstamos en curso
Acciones	- Ejecutar las transacciones ordenadas por los usuarios del sistema - Registrar en la red Ethereum la información del sistema.

Diagrama entidad relación

El diagrama representado en la figura 3.4 pretende definir mediante relaciones entre las entidades las funciones que definen el sistema.

Figura 3.4. Diagrama entidad relación



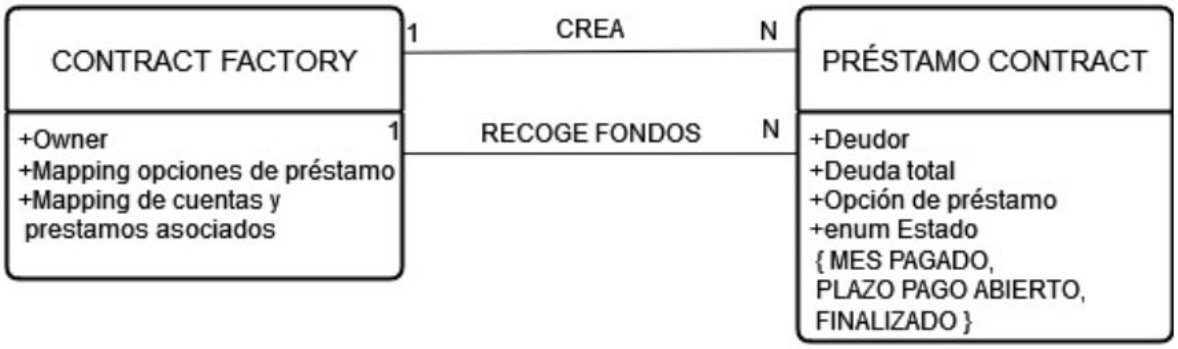
3.4. Análisis Smart contracts implicados

En esta sección se define la arquitectura de diseño utilizada para los Smart Contracts, así como los atributos (tablas 3.15 y 3.17) y funciones (tablas 3.16 y 3.18) de cada uno de ellos.

3.4.1. Diagrama Entidad - Relación de Smart Contracts

Aunque no sean entidades del sistema, para explicar detalladamente la arquitectura de de diseño se define otro diagrama entidad - relación en la figura 3.5

Figura 3.5. Diagrama entidad relación que representa las relaciones y funciones entre los Smart contracts.



Contract Factory

En la figura anterior, 3.5, también define gráficamente el patron de diseño utilizado en el desarrollo de los Smart Contracts, a este patrón se le denomina *'Contract Factory'*.

El diseño de *'Contract Factory'* encaja perfectamente en el desarrollo de este proyecto ya que nos permite crear los distintos contratos de préstamo de forma dinámica y automática. En esta arquitectura, un contrato principal (el *Factory*) actúa como intermediario recibiendo las solicitudes y generando nuevos contratos (*contratos préstamo*) según las necesidades de los usuarios o la aplicación. Dentro de este contrato se definen los distintos tipos de préstamos que pueden ser contratados por los usuarios, asi como las distintas funcionalidades de gestión y monitorización. Para ello este contrato cuenta con varibles como:

Listado 3.1. Variable que contiene los prestamos producto

```
1    mapping(address => PrestamoCursando[])  prestamosCursando;
2    address[] clientes;
```

En el fragmento de código del listado 3.1 se observa la variable *prestamosCurando* perteneciente al Smart Contract principal *Contract Factory*, la cual guarda en la red Blockchain el conjunto de direcciones correspondientes a un contrato tipo *Préstamo* pertenecientes al sistema.

Por otra parte, cada contrato creado por *Contract Factory* , es decir cada *Contrato Préstamo* mantiene información específica como la dirección del prestatario, la deuda restante, el número de cuotas pendientes, el estado del préstamo o el porcentaje de penalización aplicable, tal y como podemos ver en 3.2

Listado 3.2. Variables contrato préstamo

```
1            uint index;
2            enum State { PAGADO, ABIERTO, FINALIZADO }
3            State state;
4            address prestatario;
5            address entidadFinanciera;
6            address payable prestamosFactory;
7            uint256 cuotasRestantes;
8            uint256 cantidadPrestada;
9            uint256 deudaRestante;
10           uint256 cuotaMensual;
11           uint256 penalizacion;
12           uint256 ultimoCheckeo;
```

3.4.2. Análisis de Smart Contracts

Contract Factory

El contrato *Contract Factory* centraliza la gestión de los tipos de préstamo disponibles y el registro de los préstamos asociados a cada usuario. Para ello, mantiene un conjunto de atributos que identifican al propietario del sistema, definen las condiciones de los productos y relacionan a los usuarios con sus contratos de préstamo. La tabla 3.15 recoge estos atributos.

Tabla 3.15. Descripción de atributos del Contrato Contract Factory

Nombre	Tipo de dato	Descripción
Owner	Tipo Address	Dirección de la red Ethereum perteneciente a la propietaria del sistema.
Tipos de préstamo	Mapping ((Uint =>Struct) "tipo prestamo")	Mapping que guarda los distintipos tipos de préstamo contratables por el usuario. Esta variable solo puede ser editada si se hace identifacado desde la dirección de Owner.
TipoPréstamo	Struct	Conjunto de variables que definen un tipo de préstamo(Importe, intereses, deuda total, penalización de impago y cuotas)
Préstamos en curso	Mapping (Address =>Address[])	Mapping que relaciona la dirección de un usuario con las direcciones de sus préstamos en curso. Esta variable solo puede ser editada si se hace identifacado desde la dirección de Owner.

A continuación, la tabla 3.16 presenta las funciones del contrato *Contract Factory*, sus parámetros de entrada y salida y una descripción de su finalidad.

Tabla 3.16. Descripción de funciones del Contrato Contract Factory

Nombre	Parámetro entrada	Parámetro salida	Descripcion
crearTipoPrestamo	Importe, interesTotal, penalización impago, número cuotas		Añade un nuevo tipo de préstamo al mapping tipos de préstamo.
crearPrestamo	Tipo de préstamo, Address de usuario		Añade un nuevo tipo de préstamo al mapping tipos de préstamo
chequeoMensual			Actualiza los estados de los contratos y calcula la nueva cuota mensual
consultarContratos		Mapping de préstamos en curso	Devuelve la listado de préstamos en curso
recuperarFondos	Cantidad		Transfiere unidades desde el *Contract Factory* a la dirección del Owner
invertirFondos	Cantidad		Transfiere unidades *Contract Factory*.
consultarPrestamosUsuario	Dirección de usuario a consultar	Conjunto de de Direcciones de *Contráto Préstamo* asociados a un mismo usuario	

Préstamo Contract

El Smart Contract `PrestamoCursando` representa un préstamo individual contratado por un usuario. Almacena las condiciones del préstamo, permite el pago de las cuotas

y mantiene actualizado su estado tras el chequeo de mensualidad. A continuación, en la tabla 3.17 se describen los atributos implicados.

Tabla 3.17. Descripción de atributos del Contrato Préstamo Contract

Nombre	Tipo de dato	Descripcion
ID	Int	Identificador único
Estado	ENUMERADO	CUOTA PAGADA, PLAZO ABIERTO, FINALIZADO
CuotasRestantes	Int	Cantidad de pagos mensuales restantes
Prestatario	Tipo Address	Dirección del usuario que ha contratado el préstamo
Deuda Restante	Float	Deuda restante del préstamo
Cuota Mensual	Float	Cantidad de la deuda a pagar mensualmente
Porcentaje penalización	Int	Porcentaje de penalización a la deuda en caso de impago

El encargado de las operaciones de actualización de importes del *Contrato Préstamo* es el propio Smart Contract a traves de las funciones explicadas en la figura 3.18

Tabla 3.18. Descripción de funciones del Contrato Préstamo Contract

Nombre	Parámetro entrada	Parámetro salida	Descripcion
pagoCuota			Transfiere al contrato la cuota previamente calculada, este importe es transferido directamente al contract factory después de dejar reflejado que se ha pagado la cuota.
aplicarPenalización		Nuevo valor de deuda	Añade el porcentaje de penalización del contrato a la deuda total
calculoCuotaMensua		Valor de la cuota mensual	A partir de la deuda restante y el numero de cuotas restantes calcula el la cuota.

3.4.3. Decisiones de implementación en el sistema

Durante el desarrollo de los Smart Contract se han tomado diversas decisiones técnicas reservar responsabilidades a la red, controlar la identificación del sistema o realizar correctamente el registro de las transacciones. A continuación, se presentan las principales decisiones adoptadas:

Controles de acceso reflejados en la propia red Al contar cada *Contrato Préstamo* con su propia información, se garantiza que cada préstamo es independiente y el sistema de control de acceso puede ser gestionado de manera casi autónoma si está reflejado en la Blockchain.

Se puede ver un ejemplo de gestión autónoma en la función 3.3 del *Contrato Préstamo*, en la que a través de las lineas que comienzan por `require`, aseguramos que:

- El prestatario sea el único que puede realizar el pago de la mensualidad.
- El importe enviado es exactamente el valor de la cuota mensual
- El estado del préstamo permite realizar el pago.

Listado 3.3. Función para pagar mensualidad

```
1            function pagarMensualidad() payable public {
2                require(msg.sender == prestatario, "Only borrower");
3                require(msg.value == cuotaMensual, "Send exact debt
                     value");
4                require(state == State.ABIERTO, "Payment period not
                     open");
5
6                state = State.PAGADO;
7                deudaRestante -= cuotaMensual;
8                cuotasRestantes--;
9
10               if(cuotasRestantes == 1){   cuotaMensual =
                     calculoCuotaMensual(); }
11
12               if(cuotasRestantes == 0){  state = State.FINALIZADO;
                      }
13               transferirFondosAFactory(msg.value);  }
```

Contrato préstamo *abstracto* Para que la creación de varios tipos de *contrato prestamo* sea dinámica y se puedan definir desde la dApp se ha optado por crear un único contrato que pueda ser parametrizado en el momento de su creación. En el contrato *factory* se define un *struct* 3.4 que contiene los parámetros de cada tipo de préstamo, como el importe, los intereses, la penalización por impago y el número de cuotas.

Listado 3.4. struct TipoContrato

```
1                struct TipoContrato {
2                    uint256 index;
3                    uint256 porcentajeInteres;
```

```
4                   uint256 cantidad;
5                   uint256 deudaTotal;
6                   uint256 cuotas;
7                   uint256 penalizacionImpago;
8               }
```

Cuando un usuario solicita un préstamo, se pasa el tipo de préstamo seleccionado al constructor del *Contract Préstamo* 3.5, que inicializa sus variables con los valores correspondientes.

Listado 3.5. struct TipoContrato

```
1               constructor(uint256 _index, address _prestatario,
                   address _entidad, uint256 _cuotasRestantes,
                   uint256 _cantidad, uint256 _deudaTotal, uint256
                   _penalizacion){
2                   index = _index;
3                   state = State.ABIERTO;
4                   prestatario = _prestatario;
5                   entidadFinanciera = _entidad;
6                   prestamosFactory = payable(msg.sender);
7                   cuotasRestantes = _cuotasRestantes;
8                   cantidadPrestada = _cantidad;
9                   deudaRestante = _deudaTotal;
10                  penalizacion = _penalizacion;
11                  cuotaMensual = calculoCuotaMensual();
12                  ultimoCheckeo = block.timestamp;
13              }
```

Direcciones como identificación

En un sistema basado en Ethereum no es necesario generar un identificador centralizado para identificar a un usuario. La dirección pública de su cartera constituye un identificador único dentro de la red. Cada solicitud de producto solicitada al Smart Contract factory genera un nuevo contrato préstamo en la red con su correspondiente dirección en ella quedando registrado en la variable *prestamosCursando*. Esta variable

relaciona la dirección del usuario solicitante con un *array* de direcciones de los contratos de préstamo que tiene en curso. De esta forma se puede registrar las direcciones de los contratos creados y proporcionar mecanismo centralizado para su administración, por ejemplo a través de la función 3.7

Listado 3.6. Función que devuelve los contratos de un usuario

```
1     function verContratos(address _prestatario) public view returns
          (PrestamoCursando[] memory){
2         require((msg.sender == owner)||(msg.sender == _prestatario),
              "Top Secret");
3         return (prestamosCursando[_prestatario]);
4     }
```

Representación de cantidades monetarias

Debido a que la versión estable del lenguaje *solidity* para los Smart Contracts en el momento del desarrollo del proyecto puede tener ciertos errores a la hora de tratar decimales, la mejor decisión es tratar todas las cifras monetarias como enteros de la únidad mínima (Wei) evitando así los distintos problemas que puedan surgir con los decimales

Reducción del consumo de gas

Siempre que una operación no requiere necesariamente modificar el estado de la blockchain, esta se ejecuta en la aplicación web del cliente. De esta manera se evita utilizar Gas para operaciones puramente relacionadas con la visualización de información, como el recorrido de los contratos de un usuario para mostrar su estado. En este sistema se recupera la información de todos los contratos 3.7 y es la aplicación web la que filtra y muestra la información de cada contrato 3.8, El primer fragmento de código recupera la infromación de la red y el segundo la muestra por pantalla al usuario.

Listado 3.7. Recuperacion de informacion de contratos

```
1         async iniciarVistaContratados(){
2     this.direccionesContratados = await this.factoryContract.methods
```

```
        .verContratos(localStorage.getItem('userAddress'))
            .call({ from: localStorage.getItem('userAddress')});

        this.direccionesContratados.forEach(async (direccion: any) => {
        var prestamoCursando = new this.web3.eth.Contract(PRESTAMOCURSANDO.abi, direccion);
        var contratado = await prestamoCursando.methods.mostrarInfo().call({from: localStorage.getItem('userAddress')});
        contratado._ultimoCheckeo = this.secondsToDateString(contratado._ultimoCheckeo);
        contratado._state = this.contratoStateToString(contratado._state);
        this.listaContratados.push(contratado);
        });
  }
```

Listado 3.8. Muestra de informacion de contratos

```
<ion-col *ngFor="let prestamo of listaContratados;" size="3.1"  >
      <ion-card >
        <ion-card-content >
          <ion-list >
            <ion-item > <ion-label  class = "ion-text-center" >Estado : {{ prestamo._state.toString() }}</ion-label> </ion-item>
            <ion-item > <ion-label  class = "ion-text-center" >Deuda Restante: {{ prestamo._deudaRestante.toString() }}</ion-label> </ion-item>
            <ion-item > <ion-label class = "ion-text-center" >Cuotas Restantes: {{ prestamo._cuotasRestantes.toString() }} </ion-label> </ion-item>
            <ion-item > <ion-label class = "ion-text-center" > Cantidad Próxima Cuota: {{ prestamo._cuotaMensual.toString() }} </ion-label> </ion-item>
            <ion-item > <ion-label class = "ion-text-center">Fin plazo acutal: {{ prestamo._ultimoCheckeo }}</ion-label > </ion-item>
            <ion-item> <ion-label class = "ion-text-center">
```

```
                Penalización por Impago: {{ prestamo.
                _penalizacionImpago.toString() }}%</ion-label> </ion-
                item>
            </ion-list>
            <ion-button (click)="pagarCuota(prestamo._direccion,
               prestamo._cuotaMensual)">PAGAR CUOTA ({{prestamo.
               _cuotaMensual.toString()}})</ion-button>
          </ion-card-content>
        </ion-card>
</ion-col>
```

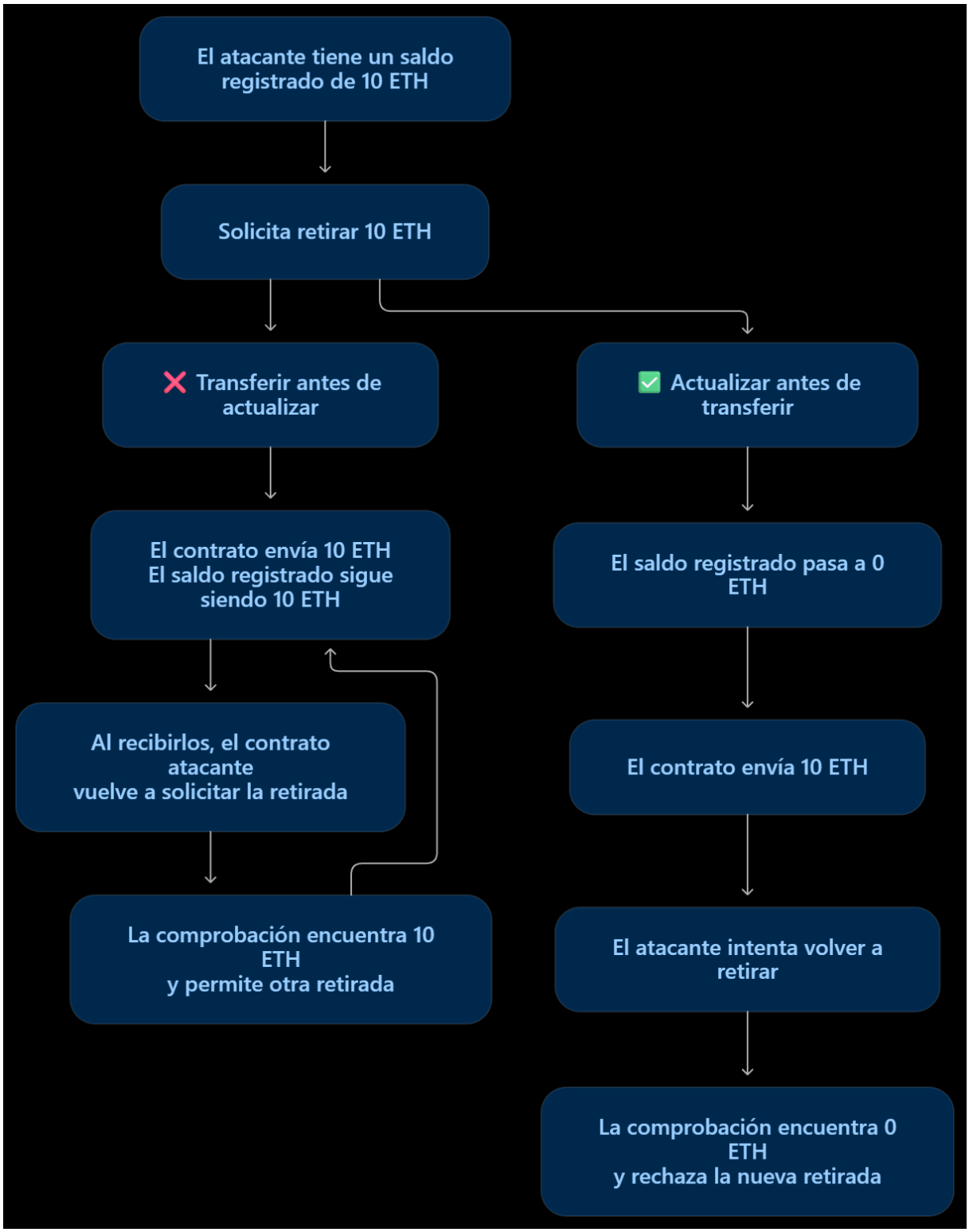
Registro del estado antes de realizar transferencias

En solidity es una buena práctica actualizar realizar un conjunto de validaciones en las condiciones de la operación y modificar el estado de las variables y saldos de las cuentas antes de realizar transferencias de fondos. Existe un tipo de ataque llamado *Ataque por reentrada*. En este ataque el riesgo se produce cuando un Smart Contract transfiere fondos a otro contrato y este aprovecha la ejecución de su código para volver a invocar una función del contrato original antes de que termine la operación inical. Si todavía no se han actualizado las variables de saldos, el atacante podría repetir una retirada utilizando un estado de las variables desactualizado y llegar a extraer fondos indebidamente.

Se puede ver como se realiza en este proyecto una transferencia de saldo en el fragmento de código 3.3.

Para un mayor entendimiento de este tipo de ataques se añade la figura **??** con un gráfico poniendo un ejemplo con una transferencia de 10 ETH.

Figura 3.6. Diagrama de un ataque de reentrada



3.5. Validación de resultados

3.5.1. Validación a través del uso de la aplicación del sistema

Tras el desarrollo del sistema de préstamos descentralizado, es necesario comprobar si cumple con los objetivos planteados y cubre los casos de uso definidos. Para ello, se ha realizado manualmente un flujo de uso de la interfaz de usuario y se presentan pruebas mediante capturas de pantalla de la interfaz y registros en la herramienta *Etherscan* para validar la ejecución de als transacciones y el correcto funcionamiento de los Smart Contract en la Blockchain de Ethereum. Para comprender las siguientes imágenes es importante conocer los siguientes conceptos:

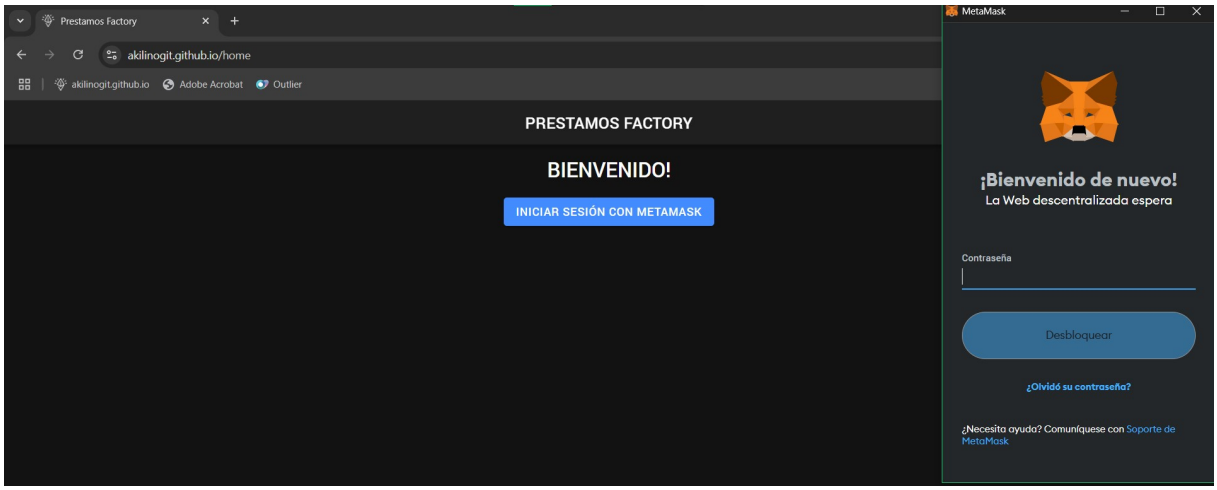
1. **Wei:** En la red Ethereum es la unidad más pequeña de valor para su criptomoneda ETH. Un ETH está compuesto por un total de 10^{18} wei.
2. Las cadenas de caracteres que comienzan por `'0x'` en la red Ethereum están reservadas como direcciones que pueden apuntar a usuarios, entidades o Smart Contracts.

El OP **??** consiste en la unificación de las distintas funcionalidades del proyecto en una interfaz accesible a cualquier sujeto independiente de la formación. Los resultados se podrán ver a lo largo de la sección y se podrá apreciar que, aunque la interfaz si reúne todas las funcionalidades del sistema, no se ha recalcado en la accesibilidad e intuitividad. Durante el desarrollo del proyecto se ha decidido aportar más esfuerzo en la funcionalidad dejando la evolución de la interfaz de usuario a un estado mas *amigable* para trabajo futuro.

Para la validación de resultados del OP **??** se pueden evalúar los distintos objetivos secundarios:

1. **OS ??:** Una de las preocupaciones que puede suponer para el desarrollo de un sistema de préstamos en una Blockchain es la correcta identificadión de los usuarios cuando la naturaleza de la red es puramente anónima. Como se ve en la figura 3.6 este proyecto hace uso de la plataforma Metamask como agente identificador externo para acceder a la aplicación.

Figura 3.7. Ejemplo de login con Metmask



2. La utilización de Metamask no solo nos sirve para delegar la identificación sino también para delegar en una empresa externa la información que identifica a nuestros clientes, de forma que no hace falta grabarla en la red. Esto nos ayuda con el el OS **??** el cual busca este punto de unión. Gracias a Metamask aunque existan cientos de operaciones económicas reflejadas en la red, sólo es posible comprobar la identidad de los participantes de estas operaciones a través de Metamask y las claves privadas de cada usuario.

3. En el OS **??** 2 se busca la eficiencia en la información y las funciones del sistema para evitar el alto consumo de energía y Gas a la hora de ejcutar las transacciones en la red:

   Todas las funciones que tienen relación con la red son funciones muy atómicas y sencillas evitando los grandes procesamientos de datos en la red aunque es posible que la función que se encarga de comprobar los pagos de los contratos de forma mensual, puede suponer una alta inversión por parte de la entidad financiera en caso de que tengo un volumen alto de préstamos, lo cual nos lleva a una posible actuación futura para limitar la cantidad de procesamiento en esta acción.

4. Haciendo referencia al OS **??** se adjuntan las siguientes imágenes del panel de control de la entidad financiera en el que se muestran 3 contratos de un usuario en curso antes y después de la comprobación mensual de los pagos de las cutoas. En ellas se pueden observar los distintos detalles:

   a) El balance de la entidad despés del pago de cuota del usuario aumenta correctamente

b) Uno de los contratos no ha pagado correctamente la cuota y se le ha aplicado auomáticamente la penalización por impago.

c) Uno de los contratos correctamente pagado ha reducido su deuda restante y el número de cuotas restantes.

d) En el caso del contrato que solo tenía una cuota restante, ya no se muestra en la dApp ya que no es necesario replicar más esta información.

Estas comprobaciones se pueden ver reflejadas con el cambio de información correspondiente en la figura que muestra la información previa a la comprobación mensual 3.7 y tras la comprobación mensual en la figura 3.8

Figura 3.8. Información de contratos previa a la comprobación mensual en el recuadro amarillo y balance de la entidad financiera en el recuadro blanco.

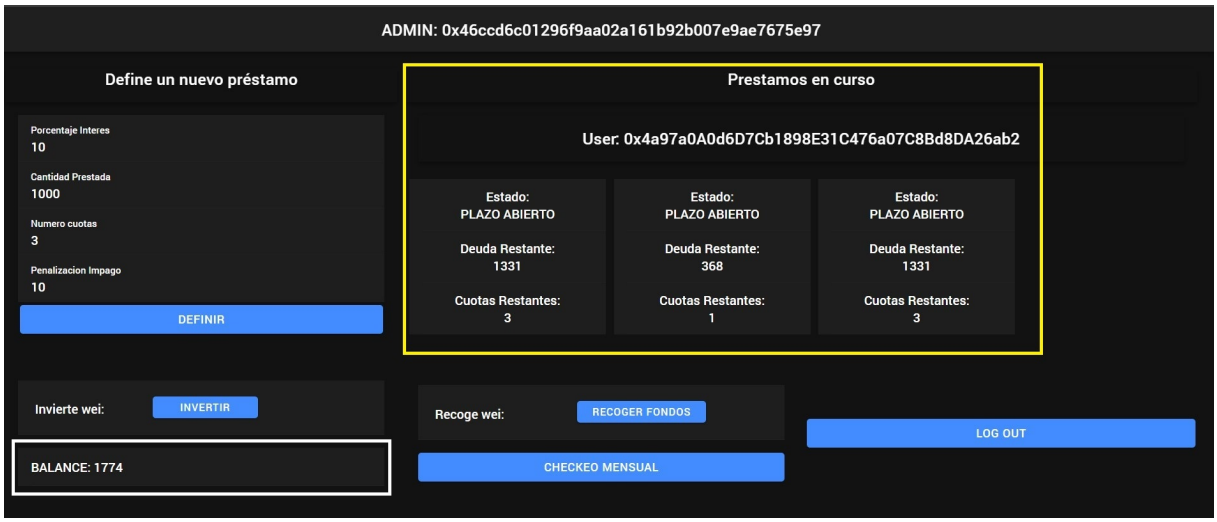
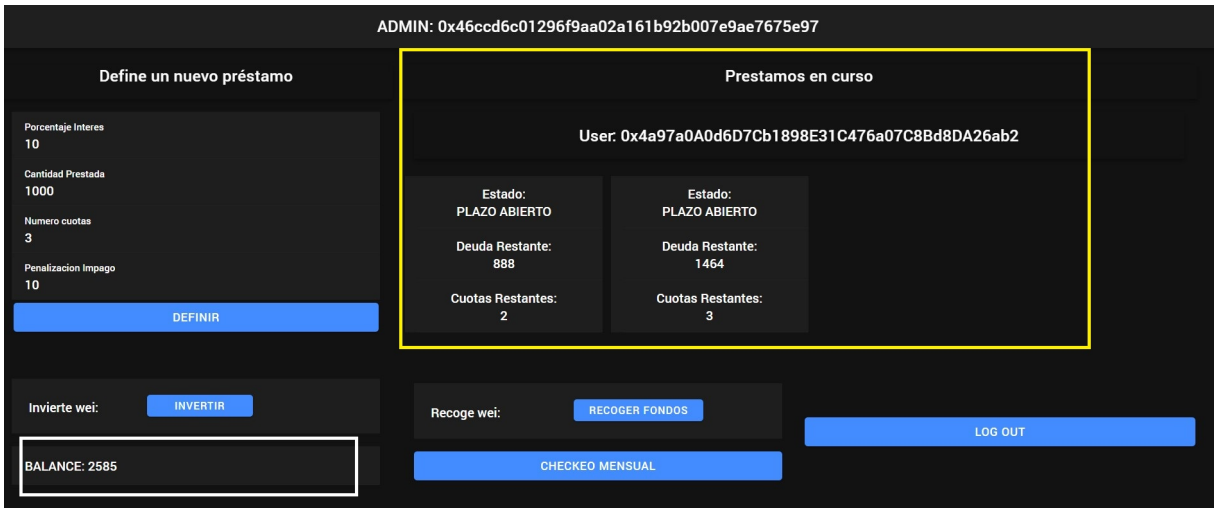


Figura 3.9. Información de contratos posterior a la comprobación mensual en el recuadro amarillo y balance de la entidad financiera en el recuadro blanco.



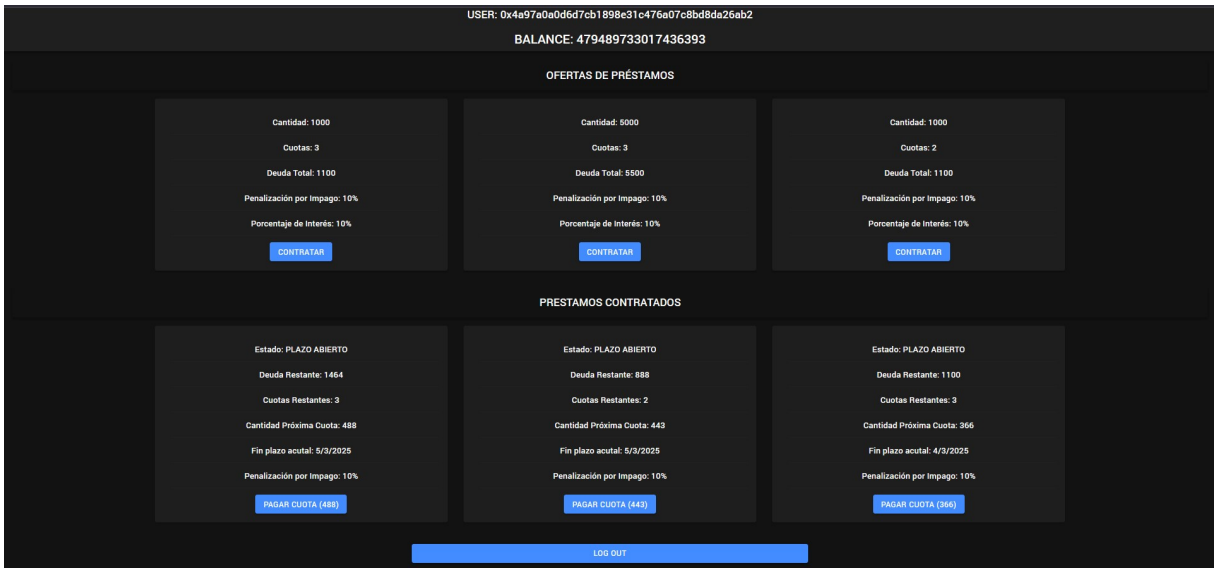
Las últimas características declaradas junto con la posibilidad de que la organización del sistema pueda recibir los Wei del balance del sistema a través de la función *Recoger Fondos* nos llevan al alcance del OS**??** el cuál busca encontrar la liquidez en este sistema.

5. En el OS **??** se contempla la posibilidad de personalizar los contratos accesibles para los distintos usuarios, esto se puede clasificar como inalcanzable para este proyecto debido a la necesidad existente en procesar grandes cantidades de datos de clientes para poder crear un algoritmo y un sistema que cubra esta necesidad.

   De momento existe la posibildad de definir nuevos préstamos contratables en el panel de control de la entidad financiera.

Para terminar de validar el OP **??** se adjunta captura de pantalla del panel de control del usuario en la figura 3.9 demostrando una interfaz unificada para el usuario.

Figura 3.10. Panel de control usuario.



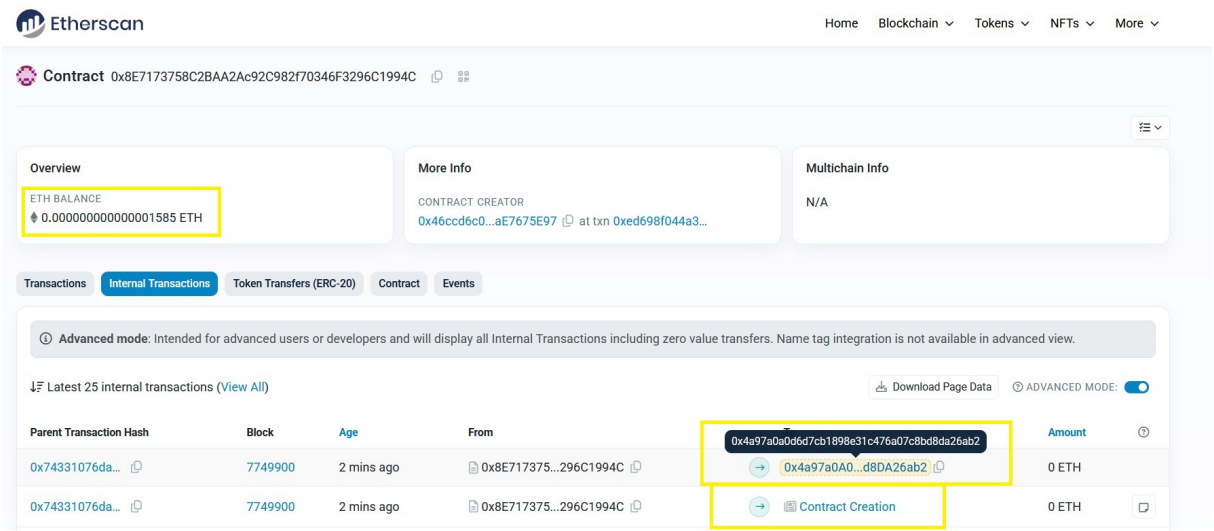
Como última prueba de validación manual de este proyecto se realiza un ejemplo de contratación de préstamo adjuntado distintas capturas de la web Etherscan Sepolia con las que se puede comprobar distintos items relacionados con el proceso que corroboran la veracidad del préstamo a través de la red Sepolia.

Ejecución de préstamo de 1000 Wei **Contrato principal:** En la anterior figura3.8 se puede comprobar que el balance del contrato principal es de 2585 Wei.

Balance del usuario: Después de la contratación del usuario, se puede ver a través de Etherscan Sepolia, tal y como se muestra en la figura 3.10, que:

- El balance del contrato ha disminuido 1000 Wei
- Se ha creado una nueva transacción relacionada con el contrato principal tipo *Contract Creation* el cual es el contrato generado para el préstamo.
- Aunque debido al redondeo de la página aparezca otra transacción de 0 ETH, viendo el balance se puede comprobar que es de 1000 Wei y han sido transferidos a la dirección del usuario .

Figura 3.11. Información Etherscan del registro en la red SepoliaETH del SmartContract principal

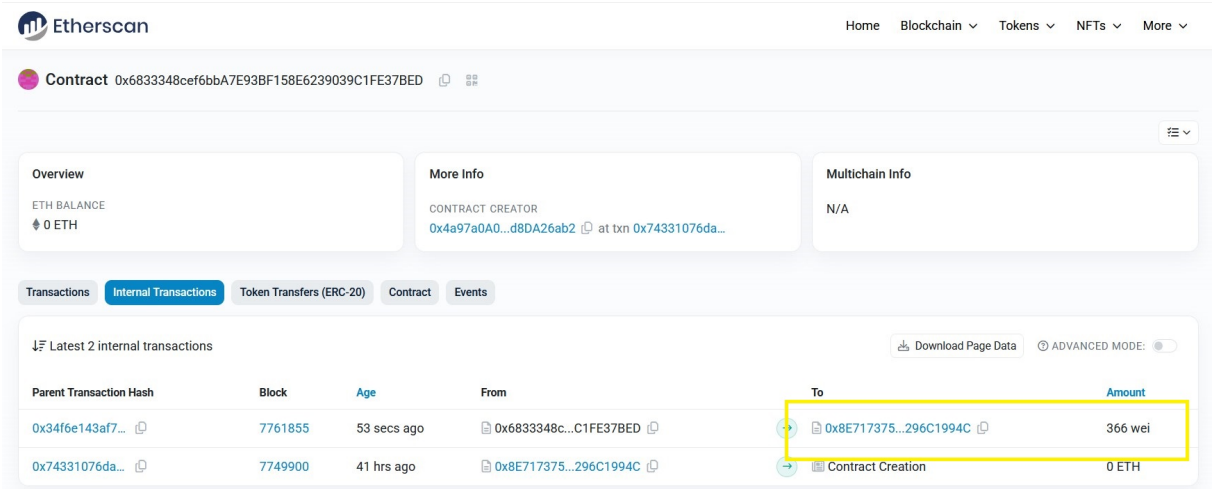


Se adjunta también otra captura de pantalla en la figura 3.11 de la información de la instancia generada del Smart Contract *Prestamo Contract*

En la figur 3.11 también se pude observar dentro del recuadro amarillo la primera transferencia recibida por parte del usuario con la cuota mensual. Esta cantidad de 366Wei coincide correctamente con el atributo *Cantidad próxima cuota* del tercer préstamo contratado en el panel de usuario de la figura 3.9.

Con esta comprobación de función de pago en Etherscan Sepolia se da por validado que todas las funciones del proyecto se reflejan en la red otorgando veracidad e inmutabilidad al proyecto.

Figura 3.12. Registro en Sepolia en el que se puede ver una transacción o comprobante de pago Etherscan



3.5.2. Mocha test

Ya se ha visto el correcto funcionamiento de la aplicación a traves de la interfaz de usuario y la verficación de las transacciones en la red. Pero para garantizar el correcto funcionamiento de los smart contracts en situaciones distintas al correcto uso de la aplicación, se han desarrollado distintos test unitarios utilizando la herramienta Mocha test Framework

Estos tests tienen como objetivo:

1. Comprobar el funcionamiento de los contratos inteligentes.
2. Verificar que las operaciones de préstamos funcionan correctamente.
3. Detectar errores en condiciones normales y situaciones no permitidas.
4. Comprobar las restricciones de acceso y los cambios de estado.

Red de pruebas local Para evitar utilizar una red pública se han desplegado los contratos sobre una red local utilizando la herramienta Ganache [24]. Esto evita que aparezcan transacciones de prueba exponiendo información sensible del funcionamiento del sistema en la red pública y permite la repetición de escenarios que son difíciles de reproducir en un entorno real, como que un usuario no pague su cuota mensual y se le aplique la penalización correspondiente.

Cuentas utilizadas para los test La herramienta de creación de red local virtual es la encargada de generar distintas cuentas con las que interactuar en la red. Se han utilizado 3 cuentas para realizar los test unitarios:

1. Una cuenta propietaria del sistema
2. Dos cuentas de usuario que solicitan el préstamo

Contratos inteligentes puestos a prueba Los test se han hecho sobre el motor de este sistema, es decir, sobre los Smart Contracts que definen el funcionamiento del sistema de préstamos. Estos contratos se pueden ver en el anexo A y son los ya comentados *Contract Factory* y *Prestamo Contract*

Tecnología de los test Los test se han desarrollado utilizando JavaScript y el framework Mocha y la librería Chai para las aserciones. Estos test se pueden encontrar en el anexo **??**

Desarrollo de los test: Variables previas y condiciones iniciales Previo al desarrollo puro de los test, antes de cada ejecución se deben dar las condiciones iniciales para que los test puedan ejecutarse correctamente. El proceso de preparación de los test consiste en desplegar los contratos en la red local y establecer las condiciones iniciales para cada prueba. Para ello se han realizado los siguientes pasos:

1. **Compilación en local de los smart contracts** A través de la herramienta Truffle se compilan los smart contracts previamente para que la red sea capaz de adquirir sus funciones. Esta compilación genera un archivo .json con las funciones del smart contract parametrizadas para que la red sea capaz de interactuar con él.
2. **Obtención de cuentas** Se obtienen las cuentas necesarias para realizar los test.
3. **Despliegue del contrato en la red** En el desarrollo de los test se hace uso del archivo .json de la compilación y se despliega el contrato a testear en la red local de pruebas.

Listado 3.9. Despliegue de contrato en la red local

```
1                    describe('Prestamos factory test', () => {
```

```
2
3                    ```
4                    before( async() => {
5                        accounts = await web3.eth.getAccounts();
6                        factory = await new web3.eth.Contract(
                            prestamosFactoryBuild.abi)
7                             .deploy({data: prestamosFactoryBuild
                                 .evm.bytecode.object})
8                             .send({ from: accounts[0], gas:
                                 6721975});
9
10                       console.log('contract address:'+ factory
                            .options.address);
11                       console.log('Owner: ' + accounts[0]);
12
13
14                   } );
15                   ```
```

4. **Establecimiento del estado inicial** En el caso de los test para los Prestamos en curso, se establece primero las variables previas necesarias a la contratación de un prestamo. Estas son:

   a) **Introducción de fondos** El sistema de préstamos necesita fondos previos para que se le pueda solicitar prestamos.

   b) **Creación de un tipo de préstamo** Se crea un tipo de préstamo con las características necesarias para el test.

   c) **Creación de contratos de prestamo y obtención de sus direcciones** De esta manera se obtiene la forma de acceder a los contratos sobre los que se ejecutarán los test.

Listado 3.10. Parametrización de Prestamos cursando

```
1    describe('Prestamos Cursando test', async () => {
2    //INICIALIZAR PRESTAMOS EN CURSO
3    before(async() => {
4        //Inicializar cuentas
5        accounts = await web3.eth.getAccounts();
```

```
6
7
8           //Inicializar contract factory
9           factory = await new web3.eth.Contract(
                prestamosFactoryBuild.abi)
10              .deploy({data: prestamosFactoryBuild.evm.bytecode.
                    object})
11              .send({ from: accounts[0], gas: 6721975});
12
13
14          //Invertir para poder crear nuevos contratos
15          await factory.methods.invertir().send({
16                  from: accounts[0],
17                  value: web3.utils.toHex(10000)
18              });
19          //Definir préstamo
20          await factory.methods.definirPrestamo(10,1000,2,10).send
                ({from: accounts[0],
21              gas: web3.utils.toHex(2000000)});
22          //Contratar prestamos
23          await factory.methods.contratarPrestamo(accounts[1],0).
                send({from: accounts[1],
24              gas: web3.utils.toHex(2000000)});
25          await factory.methods.contratarPrestamo(accounts[2],0).
                send({from: accounts[2],
26              gas: web3.utils.toHex(2000000)});
27
28          [prestamo1address]=await factory.methods.verContratos(
                accounts[1]).call({from: accounts[1]});
29          [prestamo2address]=await factory.methods.verContratos(
                accounts[2]).call({from: accounts[2]});
30
31          prestamo1 = await new web3.eth.Contract(
                prestamoCursandoBuild.abi,prestamo1address);
32          prestamo2 = await new web3.eth.Contract(
                prestamoCursandoBuild.abi,prestamo2address);
33
34      });
```

5. **Descripción test** El contenido de los test se pueden ver en los dos ficheros del anexo **??** y se describen a continuación: "'latex

   a) **1. Pruebas del `PrestamosFactory`**

   En `test_prestamos_factory.js` se comprueba principalmente el **funcionamiento del contrato Factory y sus restricciones**:

   1) **Despliegue del contrato**: se verifica que el `PrestamosFactory` se despliega correctamente y obtiene una dirección.
   2) **Control de acceso**: se comprueba que determinadas funciones únicamente pueden ser ejecutadas por el `owner`.
   3) **Ingreso de fondos**: se comprueba que `invertir()` incrementa correctamente el saldo del Factory.
   4) **Consulta de saldo**: se verifica que `getBalance()` devuelve el saldo real del contrato.
   5) **Estado inicial**: se comprueba que inicialmente no existen clientes, tipos de préstamo ni préstamos en curso.
   6) **Retirada de fondos**: se verifica que `recolectarFondos()` reduce correctamente el saldo del Factory.
   7) **Validación de parámetros**: se prueban valores incorrectos para los porcentajes de intereses y penalizaciones, comprobando que las operaciones son rechazadas.
   8) **Creación de tipos de préstamo**: se comprueba que `definirPrestamo()` crea correctamente un nuevo producto y genera el evento correspondiente.
   9) **Consulta de tipos de préstamo**: se verifica que los datos almacenados coinciden con los valores introducidos.
   10) **Contratación de préstamos**: se comprueba tanto la contratación correcta como situaciones en las que la operación debe fallar.
   11) **Consulta de préstamos de un usuario**: se verifica que `verContratos()` devuelve los contratos correspondientes.
   12) **Intento de eliminar un préstamo no finalizado**: se comprueba que esta operación no está permitida.

   b) **2. Pruebas del `PrestamoCursando`**

   El segundo archivo prueba el **ciclo de vida de un préstamo individual**.

   Se parte de una situación preparada con:

   1) Factory desplegado.

2) Fondos introducidos.
3) Tipo de préstamo creado.
4) Dos préstamos contratados por usuarios diferentes.

A partir de ahí se prueban:

1) **Control de acceso a la información**: un usuario diferente al prestatario no puede consultar la información privada del préstamo.
2) **Consulta de información del préstamo**: se comprueban importe prestado, deuda, cuota mensual, número de cuotas, estado, prestatario, etc.
3) **Restricción temporal**: se comprueba que no se puede realizar el chequeo mensual antes de que haya transcurrido el tiempo establecido.
4) **Pago de una cuota**: se realiza un pago y se comprueba que:
   - *a′* disminuye el número de cuotas restantes;
   - *b′* disminuye la deuda;
   - *c′* cambia el estado del préstamo.
5) **Transferencia de fondos**: se verifica que el dinero pagado vuelve correctamente al `PrestamosFactory` y que su balance es el esperado.
6) **Chequeo mensual**: se simula el paso del tiempo y se comprueba que el chequeo:
   - *a′* actualiza el estado del préstamo;
   - *b′* aplica una penalización al préstamo que no ha sido pagado.
7) **Finalización del préstamo**: se paga la última cuota y se comprueba que el préstamo pasa al estado de finalizado.
8) **Eliminación de préstamos finalizados**: después del chequeo mensual se comprueba que el contrato finalizado deja de aparecer entre los préstamos activos del usuario y que se actualiza la lista de clientes.

6. **Comprobación de los test** Tras la ejecución de los test, se comprueba que los resultados obtenidos son los esperados y que el sistema funciona correctamente. Para ello se utilizan las funciones de aserción de la librería Chai, que permiten comparar los resultados obtenidos con los esperados y generar un informe de los test realizados.

Resultados Tras el desarrollo y ejecución de los test se puede comprobar que el sistema funciona correctamente y que se cumplen los objetivos planteados. A su vez se han encontrado 2 situaciones que no se contemplaban en el diseño inicial.

1. Existe un metodo setIndex(index) a través del cual el contrato Factory crea un indice para controlar los contratos de préstamo en curso de un usuario. Esta función está publica en la red y únicamente puede ser ejecutada por el contrato PrestamoCursando, aunque si una atacante malicioso conociese la dirección del contrato principal y tuviese el archivo compilado del contrato puediendo acceder directamente a esta función podria simular que es el contrato PrestamoCursando y modificar el índice de los contratos de un usuario pudiendo quedar invisibles.

2. La funcion que chequea el estado de todos los contratos del sistema se demora más segundos de los que el timeOut mocha tiene por defecto por lo que se ha tenido que aumentar.

3.5.3. Validación del proyecto después de pruebas

Durante la ejecución de las pruebas de este proyecto y la simulación de un entorno real se ha visto que un sistema como este puede llegar a niveles de producción en un entorno real. Dejando a parte la necesidad de un desarrollo futuro para añadir actores y elementos al sistema necesarios para un sistema de prestamos real ,como pueden ser las garantías de devolucion, a nivel técnico se afirma que la tecnología es viable.

3.5.4. Viabilidad económica

Tras estudiar los gastos de Gas en los registros de Etherscan Sepolia se puede comprobar que los gastos de contratar un préstamo dependen mucho del precio del Gas en el momento de contratación, el cual varía de forma muy dinámica dependiendo de la congestión de la red, en el proceso de de validación se han ejecutado transferencias de préstamo con costes desde 0,46 dólares hasta 30,49 dolares. Esto puede crear un cuello de botella de entrada de préstamos en caso de que se dependa en exceso de esta variable obligando a la gestión del sistema a retrasar contratos y operaciones cuando el coste sea muy alto o directamente pasar los costes a los clientes. Por otro lado, independientemente de esta variable, a nivel técnico se ha comprobado que en el momento en el que se comprueba si todos los préstamos han sido pagados o no, la cantidad de procesamiento aumenta linealmente por el número de contratos en el sistema, sumandolo a un alto coste del gas podría crear un bloqueo total del sistema por lo que sería obligatorio optimizar la forma en la que se realiza este chequeo aunque manteniendo la viabilidad del sistema.

4. Conclusiones

En este Proyecto Fin de Grado se ha desarrollado un sistema de préstamos descentralizado basado en la red Ethereum. Se puede afirmar que se ha obtenido una aplicación capaz de conectarse con una red blockchain y realizar transacciones en ella, y que gracias al registro de estas transacciones y cierta lógica aplicada a esta información, se puede componer un sistema de creación y control de préstamos digitales.

La interfaz de usuario realiza las conexiones necesarias con la red Blockchain y ordena las transacciones necesarias para cubrir las necesidades de creación y control de préstamos digitales.

Los Smart Contracts desplegados en la red ethereum definen correctamente, dentro de la red, la lógica aplicada a los atributos de las transacciones conformando los mecanismos necesarios para la funcionalidad del sistema.

Las características de seguridad e inmutabilidad que ofrecen las redes descentralizadas aplicadas a la información que el sistema guarda en la red Ethereum confirman la viabilidad técnica de las operaciones financieras de este sistema.

Durante la implementación y validación, se ha demostrado el correcto funcionamiento del sistema mediante pruebas en redes de desarrollo.

Estos resultados confirman que la solución desarrollada cumple con los objetivos iniciales los cuales intentaban crear un sistema capaz de utilizar la tecnología disponible en la actualidad para construir un sistema sólido, confiable, transparente y autónomo para la gestión de préstamos respaldados por criptomonedas. Acercando esta tecnología al usuario común y reuniendo las acciones necesarias en una única pantalla de control.

Finalmente, se concluye que la adopción de tecnologías DeFi y contratos inteligentes tiene el potencial de transformar el acceso a servicios financieros, reduciendo costos y aumentando la inclusión financiera a nivel global. Sin embargo, persisten desafíos como la volatilidad del valor de las criptomonedas y los costos de transacción en Ethereum.

4.1. Trabajo futuro

En el desarrollo de este proyecto se ha podido comprobar que un sistema de prestamos descentralizado es posible, pero, para que un proyecto como este llegue a una situación de producción real es necesario realizar una serie de desarrollos para cumplir con necesidades básicas en un sistema de préstamo real.

Se considera la más importante la adopción de garantías de impago, esta carencia del proyecto se tiene en cuenta en el Caso de uso futuro CU12 : Garantía de devolución en caso de impago, aunque actualmente el sistema sea capaz de penalizar los impagos, no se garantiza recuperar el capital prestado, para ello se pueden desarrollar dos casos de uso:

- Creación de sistema de colaterales: en un futuro será de obligado cumplimiento que el usuario que solicite un prestamo añada al sistema alguna garantía, la cual puede estar en forma de NFT, y en caso de que el usuario se declare en situación de impago, este NFT pasaría automaticamente a ser propiedad del sistema
- Creación de sistema de avales: En situaciones en los que el usuario no pueda añadir esa garantía o colateral se podrá añadir al contrato un tercero capaz de añadir un colateral al contrato e incluso encargarse de las cuotas en caso de que el usuario principal no lo haga

El nuevo sistema de garantías también debera incluir un sistema de validación de solvencia, tal y como se habla en el Caso de uso futuro CU14, lo que aportará mayor solided a la garantía de devolución.

Durante los tests de funcionamiento del sistema se pudo comprobar las ineficencias de código en la red. Sobretodo evitar recorrer todos los contratos exoistentes una vez al mes para comprobar si han sido pagados o no. Para ello el sistema se puede apoyar en elementos externos a la red como una base de datos temporal en la que puede quedar reflejado cada pago de la mensualidad en red y mensualmente ejecutar las penalizaciones en la red con los contratos que no aparecen en esta base de datos, aunque sea una base de datos externa, su uso no rompe con ninguna característica en las que se apoya el sistema ya que solo sería un apoyo técnico, manteniendo siempre reflejado en la red la situación financiera del sistema.

Con menos prioridad se deberán desarrollar sitemas que utilicen Oráculo tal y como

se habla en el Caso de uso Futuro CU13, este sistema junto a la comprobación de solvencia económica pueden servir para una correcta definición de los tipos de prestamo y condiciones disponibles en el sistema.

4.2. Impactos sociales y ambientales del desarrollo de este sistema a gran escala

La adopción masiva de un sistema de préstamos descentralizado basado en blockchain y contratos inteligentes podría generar importantes transformaciones en el ámbito financiero y social. La eliminación de intermediarios permitiría un acceso más democrático a los servicios financieros, especialmente en regiones con sistemas bancarios limitados o restrictivos.

Sin embargo, este modelo también presenta retos significativos en términos de inclusión digital, regulación y sostenibilidad ambiental.

4.2.1. Impactos sociales

Sin embargo, la reducción de intermediarios no garantiza por sí sola una mayor inclusión financiera. Su impacto social dependería de las condiciones de acceso al crédito, de las competencias digitales de los usuarios y de los mecanismos de protección disponibles [33].

1. La posibilidad de otorgar préstamos sin la necesidad de bancos tradicionales podría beneficiar a millones de personas no bancarizadas en economías emergentes. Sin embargo, la Organización para la Coperación y el Desarrollo Económico (OCDE) señala que el acceso a estos servicios dependería del conocimiento en el uso de criptomonedas, Wallet digitales y plataformas DeFi y lo que podría aumenter la brecha digital entre quienes dominan estas herramientas y quienes no. A demás, la Organización para la Coperación y el Desarrollo Económico (OCDE) también señala que la responsabilidad del usuario sobre sus propios activos dificultan la utilización de estos. Por tanto, el acceso técnico a una plataforma no equivale a una inclusión financiera efectiva. En el sistema desarrollado, sería necesario faci-

litar la comprensión de su funcionamiento y del uso de una Wallet para reducir estas barreras [33].

2. La descentralización del sistema podría entrar en conflicto con regulaciones financieras existentes. Gobiernos y bancos centrales podrían imponer restricciones al uso de plataformas DeFi, especialmente en países donde el control financiero es una herramienta de política económica. Además, la anonimidad en las transacciones podría plantear problemas relacionados con el financiamiento ilícito si no se establecen mecanismos de verificación de identidad adecuados.

3. La automatización de los préstamos a través de contratos inteligentes podría reducir la demanda de intermediarios financieros, afectando a trabajadores del sector bancario y generando resistencia por parte de instituciones tradicionales. Sin embargo, también abriría oportunidades en nuevas áreas tecnológicas, creando demanda de profesionales en blockchain, ciberseguridad y programación de contratos inteligentes.

4. **Acceso efectivo al crédito.** El BIS advierte que muchos préstamos DeFi requieren aportar garantías cuyo valor supera el importe solicitado. Esta exigencia limita su utilidad para personas que carecen de patrimonio previo y necesitan financiación [34].

4.2.2. Impactos ambientales

1. Ethereum, aunque ha reducido su impacto ambiental tras su transición de Proof of Work (PoW) a Proof of Stake (PoS), sigue dependiendo de una infraestructura global de nodos y validadores que consumen energía. A gran escala, este sistema de préstamos podría contribuir al aumento de la demanda energética de los centros de datos y redes descentralizadas, lo que requeriría soluciones de eficiencia energética y compensación de emisiones.

Si bien un sistema de préstamos descentralizado tiene el potencial de democratizar el acceso financiero y reducir costos, también presenta desafíos sociales y ambientales que deben abordarse antes de una implementación a gran escala. Para mitigar estos impactos, sería fundamental desarrollar estrategias de educación digital y establecer mecanismos de regulación equilibrados para mejorar la eficiencia energética del sistema.

Solo con un enfoque responsable y sostenible se podrá garantizar que este tipo de innovación beneficie a la sociedad sin comprometer el bienestar ambiental.

4.2.3. Objetivos de desarrollo sostenible. Agenda 2030

La Agenda 2030 para el Desarrollo Sostenible es un plan global adoptado en 2015 por los Estados Miembros de la ONU con el objetivo de erradicar la pobreza, proteger el planeta y garantizar la prosperidad para todas las personas. Para lograrlo, se establecieron 17 Objetivos de Desarrollo Sostenible (ODS), que abarcan áreas clave como la educación, la igualdad de género, la acción climática, la innovación y el crecimiento económico, entre otras.

Estos objetivos buscan equilibrar el desarrollo social, económico y ambiental, promoviendo un modelo sostenible que asegure el bienestar de las generaciones actuales y futuras. Además, llaman a la colaboración entre gobiernos, empresas y la sociedad civil para alcanzar metas concretas antes del año 2030.

En la figura 4.1 se pueden ver estos objetivos:

Figura 4.1. Objetivos de desarrollo sostenible de la agenda europea para 2030



Se puede concluir que este proyecto tiene un gran potential de influir positivamente en los objetivos relacionados en el ámbito financiero como los ODS 8 y 9 ya que supone la creación de una nueva infraestructura en el desarrollo de actividades económicas favoreciendo nuevas oportunidades de crecimiento económico.

El desarrollo de plataforma DeFi tiene un gran potencial en la reducción de desigualdades y la aplicación de justicia social (ODS 10), la cuestión es que el desarrollo de este potencial viene directamente correlacionado con el uso que se le dé a esta tecnología pudiendo desarrollarse en una dirección opuesta a estos ODS, si no se combate la brecha digital y se evita la creación de barreras articifiales pueden aumentar por parte de los desarrolladores de esta tecnologías, las desigualdades y la injusticia social aumentarían todavía mas en vez de reducirse.

Algo parecido pasa con el ODS 16: la transparencia de las transacciones en un entorno descentralizado suponen una clara exposición de injusticias o malversaciones realizadas en la red aunque esta característica pierde toda valided cuando se hace un mal uso de la justicia y suceden eventos en los que se aprecia que «la justicia NO es igual para todos»

En el apartado ambiental (ODS 11, 12 y 13), aunque, es cierto, que tras el cambio de validación de bloques en la red a través de puro procesamiento y consumo eléctrico (PoW) por un sorteo en base a las aportaciones de capital en la red (PoS) se redujo de manera drástica el consumo de electricidad, sigue habiendo otros problemas como la gestión de residuos de la infraestructura de la red. Aunque volvemos a lo mismo que los apartados anteriores, siempre se puede utilizar el aumento de productividad y capital en paliar los efectos adversos que este sistema incide en el ecosistema, todo depende del uso que se quiera dar a la tecnología.

A. Código fuente de los Smart Contracts

En este anexo se presenta el código fuente de los contratos inteligentes desarrollados para el sistema de préstamos descentralizado basado en la red Ethereum.

A.1. PRESTAMOS_FACTORY.sol

El archivo `PRESTAMOS_FACTORY.sol` implementa el patrón Factory utilizado en el sistema para gestionar la creación de nuevos contratos de préstamo. En este archivo se incluye el contrato principal `Prestamos Factory` que permite a los usuarios crear y gestionar sus préstamos de manera descentralizada. y el contrato `PrestamoCursando` que representa un préstamo en curso, con todas las funcionalidades necesarias para su gestión.

Listado A.1. Código fuente de PRESTAMOS_FACTORY.sol

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.8.0 <0.9.0;
import "@openzeppelin/contracts/utils/math/SafeMath.sol";
//SafeMath.sol no aparece con npm install, importado manualmente
contract PrestamosFactory {

    using SafeMath for uint256;

    address payable private owner;
    //Posibles prestamos a contratar
    TipoContrato[] tiposContrato;

    mapping(address => PrestamoCursando[])  prestamosCursando;
    address[] clientes;
```

```
15
16      struct TipoContrato {
17          uint256 index;
18          uint256 porcentajeInteres;
19          uint256 cantidad;
20          uint256 deudaTotal;
21          uint256 cuotas;
22          uint256 penalizacionImpago;
23      }
24
25      event PrestamoDefinido(TipoContrato tipoContrato);
26      event NuevoPrestamoCursando(address indexed direccionContrato,
            address indexed prestatario);
27      event FondosRecibidos(address direccion,uint256 cantidad);
28
29      modifier onlyOwner() {
30          require(owner == msg.sender,"Only Owner");
31           _;
32      }
33
34      constructor(){
35          owner = payable(msg.sender);
36      }
37
38      receive() external payable {
39          invertir();
40      }
41
42      function invertir() public payable {
43          emit FondosRecibidos(msg.sender, msg.value);
44      }
45
46      function getBalance() public view onlyOwner returns(uint256) {
47
48          return address(this).balance;
49      }
50      function getClientes() public view onlyOwner  returns(address[]
            memory) {
51          return clientes;
52
```

```
53      }
54
55      function recolectarFondos(uint256 amount) public  onlyOwner {
56          require(address(this).balance >= amount);
57          owner.transfer(amount);
58
59      }
60
61      function verTiposContrato() public view returns (TipoContrato[]
            memory) {
62          return tiposContrato;
63      }
64
65
66      function verContratos(address _prestatario) public view returns (
            PrestamoCursando[] memory){
67          require((msg.sender == owner)||(msg.sender == _prestatario),
                "Top Secret");
68          return (prestamosCursando[_prestatario]);
69      }
70
71
72      function definirPrestamo(uint8 _porcentajeInteres, uint256
            _cantidad, uint8 _cuotas, uint8 _penalizacionImpago) public
            onlyOwner {
73
74          require(_penalizacionImpago >= 0 && _penalizacionImpago <=
                100, "La penalizacion tiene que ser un porcentaje");
75          require(_porcentajeInteres >= 0 && _porcentajeInteres <= 100,
                "El interes tiene que ser un porcentaje");
76
77
78          uint256 _deudaTotal = _cantidad + _cantidad.mul(
                _porcentajeInteres).div(100);
79
80          TipoContrato memory nuevoPrestamo = TipoContrato({
81              index: 0 ,
82              porcentajeInteres: _porcentajeInteres,
83              cantidad: _cantidad,
84              deudaTotal: _deudaTotal,
```

```
85              cuotas: _cuotas,
86              penalizacionImpago: _penalizacionImpago
87          });
88
89          tiposContrato.push(nuevoPrestamo);
90          //Guardo el index dentro del Objeto préstamo para mayor
               facilidad de acceso
91          tiposContrato[tiposContrato.length - 1].index = tiposContrato
               .length-1;
92          emit PrestamoDefinido(nuevoPrestamo);
93
94      }
95
96
97
98
99      function contratarPrestamo(address payable _prestatario, uint8
           indexTipoContrato) public {
100
101         require(msg.sender == _prestatario, "No se puede contratar en
               nombre de otros");
102
103         TipoContrato memory tipoPrestamo = tiposContrato[
               indexTipoContrato];
104         require(address(this).balance >= tipoPrestamo.cantidad,"
               Prestamo no disponible");
105
106         //Compruebo si la cuenta tiene otro conrtrato contratado
107         if(prestamosCursando[_prestatario].length == 0){
108             clientes.push(_prestatario);
109         }
110         //Guardo index para faciltar acceso
111         uint256 _indexNuevoPrestamo = prestamosCursando[_prestatario
               ].length;
112         // Genero un nuevo contrato Prestamo asociado a la dirección
               msg.sender
113         PrestamoCursando newPrestamo = new PrestamoCursando(
               _indexNuevoPrestamo, _prestatario, owner, tipoPrestamo.
               cuotas, tipoPrestamo.cantidad,
114                                                             tipoPrestamo
```

```
                                                                    .
                                                                    deudaTotal
                                                                    ,
                                                                    tipoPrestamo
                                                                    .
                                                                    penalizacionImpag
                                                                    );
115
116         //Enviar dinero a usuario
117         emit NuevoPrestamoCursando(address(newPrestamo), _prestatario
                );
118         prestamosCursando[_prestatario].push(newPrestamo);
119         _prestatario.transfer(tipoPrestamo.cantidad);
120
121     }
122
123     function  eliminarPrestamoFinalizado(address user, uint index)
            external {
124         PrestamoCursando eliminar = prestamosCursando[user][index];
125         PrestamoCursando.State state = PrestamoCursando(eliminar).
                mostrarEstado();
126         require( state == PrestamoCursando.State.FINALIZADO , "
                Contrato no finalizado");
127         //Se copia el ultimo elemento del array y se le cambia el
                índice
128         prestamosCursando[user][index] = prestamosCursando[user][
                prestamosCursando[user].length - 1];
129         prestamosCursando[user][index].setIndex(index);
130         //Se elimina el elemento copiado
131         prestamosCursando[user].pop();
132
133         //En caso de que el cliente deje de tener contratos con
                nostros le eliminamos de la lista de clientes
134         if( prestamosCursando[user].length == 0){
135             for(uint i = 0 ; i < clientes.length; i ++ ) {
136                 if (clientes[i] == user) {
137                     clientes[i] = clientes[clientes.length - 1];
138                     break;
139                 }
140             }
```

```
141             clientes.pop();
142         }
143 
144     }
145 
146     }
147 
148 contract PrestamoCursando {
149 
150     using SafeMath for uint256;
151 
152     uint index;
153     enum State { PAGADO, ABIERTO, FINALIZADO }
154     State state;
155     address prestatario;
156     address entidadFinanciera;
157     address payable prestamosFactory;
158     uint256 cuotasRestantes;
159     uint256 cantidadPrestada;
160     uint256 deudaRestante;
161     uint256 cuotaMensual;
162     uint256 penalizacion;
163     uint256 ultimoCheckeo;
164 
165     struct InfoContrato {
166         address _direccion;
167         uint256 _index;
168         address _prestatario;
169         State _state;
170         uint256 _cantidadPrestada;
171         uint256 _deudaRestante;
172         uint256 _cuotaMensual;
173         uint256 _cuotasRestantes;
174         uint256 _penalizacionImpago;
175         uint256 _ultimoCheckeo;
176     }
177 
178     //2629800 = 1 MES;
179     uint256 public constant PLAZO = 10;
180 
```

```
181     modifier onlyFinancialEntity() {
182         require(entidadFinanciera == msg.sender,"Only financial
              entity");
183          _;
184     }
185 
186     modifier onlyContractFactory() {
187         require(prestamosFactory == msg.sender,"Only contract Factory
              ");
188          _;
189     }
190 
191     modifier hasBeenOneMonth() {
192         require(block.timestamp - ultimoCheckeo >= PLAZO, "Deadline
              has not yet expired" );
193         _;
194     }
195 
196     constructor(uint256 _index, address _prestatario, address
          _entidad, uint256 _cuotasRestantes, uint256 _cantidad, uint256
           _deudaTotal, uint256 _penalizacion){
197         index = _index;
198         state = State.ABIERTO;
199         prestatario = _prestatario;
200         entidadFinanciera = _entidad;
201         prestamosFactory = payable(msg.sender);
202         cuotasRestantes = _cuotasRestantes;
203         cantidadPrestada = _cantidad;
204         deudaRestante = _deudaTotal;
205         penalizacion = _penalizacion;
206         cuotaMensual = calculoCuotaMensual();
207         ultimoCheckeo = block.timestamp;
208     }
209 
210     function mostrarInfo() external view returns (InfoContrato memory
          ){
211         require((msg.sender == entidadFinanciera || msg.sender ==
              prestatario), "Top Secret");
212 
213         InfoContrato memory infoContrato = InfoContrato({
```

```
214             _direccion: address(this),
215             _index: index,
216             _prestatario: prestatario,
217             _state: state,
218             _cantidadPrestada: cantidadPrestada,
219             _deudaRestante: deudaRestante,
220             _cuotaMensual: cuotaMensual,
221             _cuotasRestantes: cuotasRestantes,
222             _penalizacionImpago: penalizacion,
223             _ultimoCheckeo: ultimoCheckeo
224         });
225 
226         return infoContrato;
227     }
228 
229     function mostrarEstado() external view returns (State) {
230         return state;
231     }
232 
233 
234     function setIndex(uint _index) external onlyContractFactory {
235         index = _index;
236     }
237 
238     function calculoCuotaMensual() private view returns (uint256){
239         //Debido a problemas con el tratamiento de decimales de
               solidity puede resultar el caso que
240         // deudarestante/cuotasrestantes no de un numero entero
241         //Esto implica que quede deuda despues de pagar todas las
               cuotas por lo que este resto se añade a la cuota final
242         if(cuotasRestantes == 1) {
243             return  deudaRestante;
244             }
245         else {
246             return deudaRestante.div(cuotasRestantes);
247         }
248 
249     }
250 
251     function checkeoMensual() onlyFinancialEntity hasBeenOneMonth
```

```
        public {

252
253         if(state == State.ABIERTO){
254             penalizarPrestamo();
255         }
256         else if(state == State.PAGADO){
257             state = State.ABIERTO;
258         }
259         else if (state == State.FINALIZADO){
260             eliminarContratoFinalizado();
261         }
262
263         ultimoCheckeo = block.timestamp;
264
265     }
266
267     function eliminarContratoFinalizado() private {
268         PrestamosFactory(prestamosFactory).eliminarPrestamoFinalizado
               (prestatario, index);
269     }
270
271     //Numero de cuotas se mantiene
272     function penalizarPrestamo() private {
273         deudaRestante += deudaRestante.mul(penalizacion).div(100);
274         cuotaMensual = calculoCuotaMensual();
275     }
276
277     receive() external payable {
278         pagarMensualidad();
279     }
280
281     function pagarMensualidad() payable public {
282         require(msg.sender == prestatario, "Only borrower");
283         require(msg.value == cuotaMensual, "Send exact debt value");
284         require(state == State.ABIERTO, "Payment period not open");
285
286         state = State.PAGADO;
287         deudaRestante -= cuotaMensual;
288         cuotasRestantes--;
289
```

```
290
291         if(cuotasRestantes == 1){
292             cuotaMensual = calculoCuotaMensual();
293          }
294
295         if(cuotasRestantes == 0){
296          state = State.FINALIZADO;
297         }
298         transferirFondosAFactory(msg.value);
299
300     }
301
302     //Por seguridad se pone a public para evitar fondos bloqueados en
            este contrato.
303     function transferirFondosAFactory(uint256 _amount) public {
304         prestamosFactory.transfer(_amount);
305     }
306
307
308
309 }
```

B. Código fuente de las pruebas

En este anexo se incluye el código fuente de las pruebas desarrolladas sobre los smart contract.

B.1. test_prestamos_cursando.js

Este archivo contiene las pruebas destinadas a verificar el comportamiento relacionado con los préstamos cursando.

Listado B.1. Pruebas de prestamos cursando

```
const assert = require('assert');
const ganache = require('ganache-cli');
const Web3 = require('web3');
const web3 = new Web3(ganache.provider());
const prestamosFactoryBuild = require ('../build/contracts/
   PrestamosFactory.json');
const prestamoCursandoBuild = require ('../build/contracts/
   PrestamoCursando.json');


describe('Prestamos Cursando test', async () => {
    //INICIALIZAR PRESTAMOS EN CURSO
    before(async() => {
        //Inicializar cuentas
        accounts = await web3.eth.getAccounts();


        //Inicializar contract factory
        factory = await new web3.eth.Contract(prestamosFactoryBuild.
           abi)
            .deploy({data: prestamosFactoryBuild.evm.bytecode.object
```

```
            })
        .send({ from: accounts[0], gas: 6721975});


    //Invertir para poder crear nuevos contratos
    await factory.methods.invertir().send({
            from: accounts[0],
            value: web3.utils.toHex(10000)
        });
    //Definir préstamo
    await factory.methods.definirPrestamo(10,1000,2,10).send({
       from: accounts[0],
        gas: web3.utils.toHex(2000000)});
    //Contratar prestamos
    await factory.methods.contratarPrestamo(accounts[1],0).send({
       from: accounts[1],
        gas: web3.utils.toHex(2000000)});
    await factory.methods.contratarPrestamo(accounts[2],0).send({
       from: accounts[2],
        gas: web3.utils.toHex(2000000)});

    [prestamo1address]=await factory.methods.verContratos(
       accounts[1]).call({from: accounts[1]});
    [prestamo2address]=await factory.methods.verContratos(
       accounts[2]).call({from: accounts[2]});

    prestamo1 = await new web3.eth.Contract(prestamoCursandoBuild
       .abi,prestamo1address);
    prestamo2 = await new web3.eth.Contract(prestamoCursandoBuild
       .abi,prestamo2address);


    console.log('Factory address:'+ factory.options.address);
    console.log('Owner: ' + accounts[0]);
    console.log('Prestamo1: '+ prestamo1.options.address);

    console.log('--------');
    console.log('||TEST||');
    console.log('--------');

```

```
51      });
52
53      it('mostrar info desde cuenta externa', async() => {
54          try{
55              await prestamo1.methods.mostrarInfo().call({from:
                    accounts[2]});
56              assert.fail(new TypeError('Sin acceso'));
57          }catch (error){
58
59          }
60      });
61
62      it('Mostrar info', async() => {
63          var expected = {
64              _direccion: '0xe08d37dF20bEad7007f406C847c99E367AFF4ea1',
65              _index: '0',
66              _prestatario: '0x9a9d0e305D5F97b5881055E12a9240AA1C0E8E4e
                    ',
67              _state: '1',
68              _cantidadPrestada: '1000',
69              _deudaRestante: '1100',
70              _cuotaMensual: '550',
71              _cuotasRestantes: '2',
72              _penalizacionImpago: '10',
73              _ultimoCheckeo: '1702318784'};
74
75          var acutalInfo = await prestamo1.methods.mostrarInfo().call({
                from: accounts[1]});
76          assert.equal(expected._cantidadPrestada,acutalInfo.
                _cantidadPrestada);
77          assert.equal(expected._state,acutalInfo._state);
78          assert.equal(expected._cuotaMensual,acutalInfo._cuotaMensual)
                ;
79          assert.ok(acutalInfo._prestatario);
80          assert.ok(acutalInfo._direccion);
81
82
83      });
84
85      it('Chequeo mensual temprano', async() => {
```

```
86          try{
87              await prestamo1.methods.checkeoMensual().send({from:
                    accounts[0],
88                   gas: web3.utils.toHex(2000000)});
89              assert.fail(new TypeError('No ha pasado tiempo suficiente
                    '));
90          }
91          catch(error){}
92
93      });
94
95      it('Pago Mensual', async() => {
96          await prestamo1.methods.pagarMensualidad().send({from:
                accounts[1],
97              gas: web3.utils.toHex(2000000), value: web3.utils.toHex
                    (550)});
98          prestamoPagado = await prestamo1.methods.mostrarInfo().call({
                from: accounts[1]});
99
100         assert.equal(prestamoPagado._cuotasRestantes,1);
101         assert.equal(prestamoPagado._deudaRestante, 550);
102         assert.equal(prestamoPagado._state, 0);
103
104     });
105
106     it('Transferir fondos a factory', async() => {
107         //Inversion inicial : 10000 wei
108         //Cantidad prestada : dos prestamos de 1000 wei = 2000 wei
109         //Cantidad devuelta :  Una cuota pagada del prestamo1 = 550
                wei
110         //El contrato prestamo debe de tener 0 wei y el contrato
                factory 10000-2000+550 = 8550 wei
111
112         var balanceFactory = await factory.methods.getBalance().call
                ({from: accounts[0],
113              gas: web3.utils.toHex(2000000)});
114         assert.equal(balanceFactory,8550);
115     })
116
117
```

```
118     it('Chequeo mensual', async() => {
119         //Me aseguro que pasa el tiempo suficiente para volver a
               realizar el chequeo
120         await esperar10Segundos();
121         //Realizo chequeo
122         await prestamo1.methods.checkeoMensual().send({from: accounts
               [0],
123             gas: web3.utils.toHex(2000000)});
124          //Compruebo que el estado del prestamo ha pasado de 'PAGADO'
                 a 'PRESTAMO'
125         var expectedState = 1;
126         var actualState = await prestamo1.methods.mostrarEstado().
               call();
127         assert.equal(actualState, expectedState);
128 
129         //Compruebo la penalización del prestamo no pagado
130         var expectedDeudaRestante = 1000 + (1000 * 0.1);
131         var prestamoNoPagado = await prestamo2.methods.mostrarInfo().
               call({from: accounts[2]});
132         assert.equal(prestamoNoPagado._deudaRestante,
               expectedDeudaRestante);
133 
134 
135     });
136 
137     it('Eliminación contrato finalizado', async() => {
138         //Pago ultima cuota de prestamo1
139         await prestamo1.methods.pagarMensualidad().send({from:
               accounts[1],
140             gas: web3.utils.toHex(2000000), value: web3.utils.toHex
                   (550)});
141         prestamoPagado = await prestamo1.methods.mostrarInfo().call({
               from: accounts[1]});
142         //Compruebo que sus estado está en finalizado
143         assert.equal(prestamoPagado._state,2);
144         //Realizo el chequeo mensual y este llama a la función de
               eliminar
145         await esperar10Segundos();
146         await prestamo1.methods.checkeoMensual().send({from: accounts
               [0],
```

```
147             gas: web3.utils.toHex(2000000)});
148
149         //Compruebo que no existe ni el el prestamo finalizado ni su
                usuario el cual no tiene más prestamos en el sistema
150
151         var vacio = await factory.methods.verContratos(accounts[1]).
                call({from: accounts[1],
152             gas: web3.utils.toHex(2000000)});
153         //No existe contrato para la cuenta del contrato finalizado
154         assert.equal(vacio.length,0);
155
156
157         //Solo existe un cliente que es el del contrato no pagado
158         var clientes = await factory.methods.getClientes().call({from
                : accounts[0],
159             gas: web3.utils.toHex(2000000)});
160         assert.equal(clientes.length,1);
161
162     });
163
164
165
166 });
167 async function esperar10Segundos() {
168     return new Promise(resolve => {
169       setTimeout(() => {
170         resolve('Se puede realizar otro chequeo mensual');
171       }, 10000);
172     });
173   }
```

B.2. test_prestamos_factory.js

Este archivo contiene las pruebas destinadas a verificar el funcionamiento del Prestamos Factory.

Listado B.2. Pruebas del contrato PRESTAMOS_FACTORY

```
const assert = require('assert');
const ganache = require('ganache-cli');
const Web3 = require('web3');
const web3 = new Web3(ganache.provider());
const prestamosFactoryBuild = require ('../build/contracts/
   PrestamosFactory.json');
const prestamoCursandoBuild = require ('../build/contracts/
   PrestamoCursando.json');


//Test en distinto proyecto debido al conflicto de versión de web3 y
   su constructor
describe('Prestamos factory test', () => {

    before( async() => {
        accounts = await web3.eth.getAccounts();
        factory = await new web3.eth.Contract(prestamosFactoryBuild.
           abi)
            .deploy({data: prestamosFactoryBuild.evm.bytecode.object
               })
            .send({ from: accounts[0], gas: 6721975});

        console.log('contract address:'+ factory.options.address);
        console.log('Owner: ' + accounts[0]);


    } );

    it('deploys a contract', () => {
        assert.ok(factory.options.address);
    });

    it('onlyOwner', async () => {
        try{
            factory.methods.getBalance().call({from: accounts[1]});
            assert.fail(new TypeError('Only owner'));
        } catch (error) {
        }
```

```
34      });
35
36      it('invertir wei', async () => {
37          await factory.methods.invertir().send({
38              from: accounts[0],
39              value: web3.utils.toHex(10000)
40          });
41          var expected = await web3.eth.getBalance(factory.options.
                address);
42          assert.equal(expected,10000);
43      });
44
45      it('get Balance', async () => {
46          var expected = await web3.eth.getBalance(factory.options.
                address);
47          var actual = await factory.methods.getBalance().call({from:
                accounts[0]});
48          assert.equal(expected,actual);
49      });
50
51      it('get empty clientes', async () => {
52          let clientes = await factory.methods.getClientes().call({from
                : accounts[0]});
53          assert.equal(clientes.length,0);
54      });
55
56      it('recolectar fondos', async () => {
57          await factory.methods.recolectarFondos(1000).send({
58              from: accounts[0]
59          });
60          var expected = await web3.eth.getBalance(factory.options.
                address);
61          assert.equal(expected,9000);
62      });
63
64      it('get empty tipos contrato', async () => {
65          let tiposContrato = await factory.methods.verTiposContrato().
                call({from: accounts[0]});
66          assert.equal(tiposContrato.length,0);
67      });
```

```
68
69      it('get empty contratos en curso', async () => {
70          let contratosCursando = await factory.methods.verContratos(
                accounts[0]).call({from: accounts[0]});
71          assert.equal(contratosCursando.length,0);
72      });
73
74      it('get error getting contracts from diferrent account', async ()
            => {
75          try {
76              let contratosCursando = await factory.methods.
                    verContratos(accounts[1]).call({from: accounts[0]});
77              assert.fail(new TypeError('Top Secret'));
78          }
79          catch(error) {}
80
81      });
82
83      it('definir prestamo wrong porcentage penalizacion', async() => {
84          try {
85               await factory.methods.definirPrestamo(1000,1000,10,10).
                     send({from: accounts[0]});
86               assert.fail(new TypeError('Wrong porentage'));
87          }
88          catch(error) {}
89
90      })
91      it('definir prestamo wrong porcentage interes', async() => {
92          try {
93              await factory.methods.definirPrestamo(10,1000,10,10000).
                    send({from: accounts[0],
94              gas: web3.utils.toHex(20000)});
95              assert.fail(new TypeError('Wrong porentage'));
96         }
97         catch(error) {}
98
99      });
100
101     it('definir prestamo', async() => {
102         const result = await factory.methods.definirPrestamo
```

```
            (10,1000,2,10).send({from: accounts[0],
103          gas: web3.utils.toHex(2000000)});
104         const events = result.events;
105         const eventoDefinido = events['PrestamoDefinido'];
106         assert.ok(eventoDefinido);
107
108     });
109
110     it('get tipos contrato', async () => {
111
112         var expected = {
113             index: '0',
114             porcentajeInteres: '10',
115             cantidad: '1000',
116             deudaTotal: '1100',
117             cuotas: '10',
118             penalizacionImpago: '10'};
119
120         let tiposContrato = await factory.methods.verTiposContrato().
               call({from: accounts[0]});
121         assert.equal(tiposContrato.length,1);
122         assert.equal(tiposContrato[0].index, expected.index);
123         assert.equal(tiposContrato[0].deudaTotal, expected.deudaTotal
               );
124
125     });
126
127     it('contratarPrestamo from another account', async() => {
128         try {
129             await factory.methods.contratarPrestamo(accounts[1],0).
                   send({from: accounts[2]});
130             assert.fail(new TypeError('Error en dirección'));
131
132         } catch (error){
133             console.log(error.message)
134
135         }
136     });
137
138     it('contratarPrestamo from another account', async() => {
```

```
139          await factory.methods.definirPrestamo(10,999999,10,10).send
                 ({from: accounts[0],
140              gas: web3.utils.toHex(2000000)});
141
142          try {
143              await factory.methods.contratarPrestamo(accounts[1],1).
                     send({from: accounts[1]});
144              assert.fail(new TypeError('No hay suficientes fondos para
                      el prestamo'));
145
146          } catch (error){
147              console.log(error.message)
148          }
149      });
150
151      it('contratarPrestamo', async() => {
152          var result = await factory.methods.contratarPrestamo(accounts
                 [1],0).send({from: accounts[1],
153              gas: web3.utils.toHex(2000000)});
154          var eventoDefinido = result.events['NuevoPrestamoCursando'];
155          assert.ok(eventoDefinido);
156          console.log(eventoDefinido);
157          result = await factory.methods.contratarPrestamo(accounts
                 [2],0).send({from: accounts[2],
158              gas: web3.utils.toHex(2000000)});
159          eventoDefinido = result.events['NuevoPrestamoCursando'];
160          assert.ok(eventoDefinido);
161          console.log(eventoDefinido);
162
163      });
164
165      it('verContratos', async() => {
166          var contratos = await factory.methods.verContratos(accounts
                 [1]).call({from: accounts[1]});
167          assert.equal(contratos.length,1);
168          contratos = await factory.methods.verContratos(accounts[2]).
                 call({from: accounts[2]});
169          assert.equal(contratos.length,1);
170      });
171
```

```
172     it('eliminar contrato no finalizado', async() => {
173         try{
174             await factory.methods.eliminarPrestamoFinalizado(accounts
                    [1],0).send({from: contratoEliminar});
175             assert.fail(new TypeError('Deuda pendiente'));
176             } catch (error){
177                 console.log(error.message);
178             }
179     });
180
181
182     });
```

Referencias

[1] E. Dans, «*Planteando el bitcoin como alternativa monetaria*», 2024. dirección: https://www.enriquedans.com/2024/12/planteando-el-bitcoin-como-alternativa-monetaria.html

[2] A. P. Sanz, *Repositorio de este proyecto: Sistema de préstamos descentralizado basado en la red Blockchain de Ethereum*, 2024. dirección: https://github.com/AkilinoGit/akilinogit.github.io

[3] Aquilino, *Repositorio de pruebas unitarias en mochatest*, 2026. dirección: https://github.com/AkilinoGit/prestamosFactoryMochaTest.git

[4] A. V3, *Estado del protocolo Aave V3 desplegado en la red*, 2026. dirección: https://etherscan.io/address/0x87870Bca3F3fD6335C3F4ce8392D69350B4fA4E2#code

[5] AAVE, *Documentación del protocolo AAVE*, 2026. dirección: https://aave.com/docs

[6] C. III, *Estado del protocolo Compound III desplegado en la red*, 2026. dirección: https://etherscan.io/address/0xc3d688B66703497DAA19211EEdff47f25384cdc3#code

[7] Compound, *Documentación del protocolo Compound*, 2026. dirección: https://docs.compound.xyz/

[8] U. V4, *Estado del protocolo Uniswap V4 desplegado en la red*, 2026. dirección: https://etherscan.io/address/0x000000000004444c5dc75cB358380D2e3dE08A90#code

[9] Uniswap, *Documentación del protocolo Uniswap*, 2026. dirección: https://developers.uniswap.org/docs/get-started/concepts/how-uniswap-works

[10] J. L. M. T. CARLOS DOLADER RETAMAL JOAN BEL ROIG, «*LA BLOCKCHAIN: FUNDAMENTOS, APLICACIONES Y RELACIÓN CON OTRAS TECNOLOGÍAS DISRUPTIVAS*», 2017. dirección: https://www.mintur.gob.es/Publicaciones/Publicacionesperiodicas/EconomiaIndustrial/RevistaEconomiaIndustrial/405/DOLADER%2C%20BEL%20Y%20MU%C3%91OZ.pdf

[11] M. U. G. BARTOLOMEO Alejandro, «*INTRODUCCIÓN A LA TECNOLOGÍA BLOCKCHAIN: SU IMPACTO EN LAS CIENCIAS ECONÓMICAS S*», 2020. dirección: https://bdigital.uncuyo.edu.ar/objetos_digitales/15304/14.-introduccinalatecnologia.pdf

[12] Neocheck, *SepoliaFaucet*, 2019. dirección: https://www.neocheck.es/wp-content/uploads/2019/09/funcionamiento-blockchain-permisionada-800x416.png

[13] Ethereum.org. «Historia de Ethereum: fundador y propiedad.» Sitio web oficial de Ethereum, visitado 21 de oct. de 2026. dirección: https://ethereum.org/es/ethereum-history-founder-and-ownership/

[14] Solana, *Solana*, -. dirección: https://solana.com/es

[15] BNBChain, *BNBChain*, -. dirección: https://www.bnbchain.org/es-LA

[16] C. M. Cap, *Coin Market Cap*, 2026. dirección: https://coinmarketcap.com/es/

[17] Ethereum, «*Historia de Ethereum: fundador, lanzamiento y propiedad*», 2026. dirección: https://ethereum.org/es/ethereum-history-founder-and-ownership/

[18] F. Vogelsteller y V. Buterin, *ERC-20: Token Standard*, Ethereum Improvement Proposals, no. 20, nov. de 2015. dirección: https://eips.ethereum.org/EIPS/eip-20

[19] W. Entriken, D. Shirley, J. Evans y N. Sachs, *ERC-721: Non-Fungible Token Standard*, Ethereum Improvement Proposals, no. 721, ene. de 2018. dirección: https://eips.ethereum.org/EIPS/eip-721

[20] W. Radomski, A. Cooke, P. Castonguay, J. Therien, E. Binet y R. Sandford, *ERC-1155: Multi Token Standard*, Ethereum Improvement Proposals, no. 1155, jun. de 2018. dirección: https://eips.ethereum.org/EIPS/eip-1155

[21] Coinbase, *Coinbase*, -. dirección: https://www.coinbase.com/es-es/wallet

[22] RabbyWallet, *RabbyWallet*, -. dirección: https://rabby.io/

[23] Truffle, *Truffle*, -. dirección: https://archive.trufflesuite.com/

[24] Ganache, *Ganache*, -. dirección: https://archive.trufflesuite.com/ganache/

[25] Sepolia, *Sepolia*, -. dirección: https://sepolia.etherscan.io/

[26] EtherScan, *EtherScan*, -. dirección: https://etherscan.io/

[27] RemixIDE, *RemixIDE*, -. dirección: https://remix-project.org/?lang=es

[28] NodeJS, *NodeJS*, -. dirección: https://nodejs.org/es

[29] Solidity, *Solidity*, -. dirección: https://soliditylang.org/

[30] Angular, *Angular*, 2026. dirección: https://angular.dev/

[31] Ionic, *Ionic*, 2026. dirección: https://ionicframework.com/

[32] U. Europea, *Reglamento Europeo sobre los Criptoativos*, 2023. dirección: https://eur-lex.europa.eu/ES/legal-content/summary/european-crypto-assets-regulation-mica.html

[33] OECD, «The Limits of DeFi for Financial Inclusion: Lessons from ASEAN,» OECD Publishing, inf. téc., 2024. DOI: 10.1787/f00a0c7f-en dirección: https://doi.org/10.1787/f00a0c7f-en

[34] S. Aramonte, S. Doerr, W. Huang y A. Schrimpf, «DeFi lending: intermediation without information?» Bank for International Settlements, BIS Bulletin 57, jun. de 2022. dirección: https://www.bis.org/publ/bisbull57.pdf

Índice de términos

Glosario

AAVE Protocolo de finanzas descentralizadas (DeFi) de código abierto y no custodio que permite a los usuarios depositar criptoactivos para obtener rendimientos o tomarlos prestados mediante contratos inteligentes sin intermediarios. 6

App Store Tienda digital de aplicaciones.. 14

Bitcoin Red blockchain original básada únicamente en transacciones y la que mayor valor ha conservado. 7, 11–13

Blockchain Blockchain es una tecnología de registro distribuido que almacena datos en bloques encadenados de forma segura, inmutable y descentralizada. Se usa principalmente para transacciones digitales, como criptomonedas, pero también tiene aplicaciones en contratos inteligentes, logística y más.. 3–6, 8–10, 12–16, 18, 19, 22, 23, 33

criptomoneda Activo digital que utiliza técnicas criptográficas y una red blockchain para registrar y verificar transacciones sin necesidad de una autoridad central. Puede emplearse como medio de intercambio.. 14, 16

dApp Descentrelized App o aplicación descentralizada, es una aplicación que funciona sobre una red descentralizada de blockchain en lugar de un servidor central. Utiliza contratos inteligentes para gestionar su lógica y datos. 4, 5, 14–19, 27, 28

DeFi Descentrelized Finance: Sistema financiero en un entorno descentralizdo, como puede ser una red blockchain. 1–6, 13–15, 19, 57, 59, 62

Diagrama de Se cuencia-UML Diagrama que muestra cómo interactúan los objetos de un sistema en un orden temporal, representando el intercambio de mensajes entre ellos.. 23

Diagrama-UML Un diagrama UML (Unified Modeling Language) es una representación gráfica utilizada para modelar, visualizar, documentar y diseñar sistemas de software. . 23

ETH Ether, es la criptomoneda de la red Ethereum. 1, 7, 14, 15, 19, 22, 27, 30, 68

Ether Ether (ETH) es la criptomoneda nativa de la blockchain de Ethereum. Se utiliza para pagar tarifas de transacción (gas) y ejecutar contratos inteligentes dentro de la red. También actúa como un activo digital que puede ser intercambiado o utilizado en aplicaciones descentralizadas (DApps) .. 46

Ethereum Red blockchain cuya moneda principal es el ETH y es capaz de ejecutar Smart Contract. 3, 7, 12, 13, 15, 16, 18, 19, 31, 35, 40, 68

Euribor Es el tipo de interés promedio al que los principales bancos de la zona euro se prestan dinero entre sí en el mercado interbancario. Se calcula diariamente y sirve como referencia para numerosos productos financieros, especialmente para hipotecas a tipo variable en Europa. . 26

Framework Conjunto de herramientas, bibliotecas y convenciones diseñadas para facilitar el desarrollo de aplicaciones, proporcionando una estructura base que simplifica y organiza el código. 8, 17

Gas Cantidad de ETH necesario para la ejecución de una transacción. Este valor varía. . 3, 22, 56

hash Un hash es un valor único generado por una función hash a partir de un conjunto de datos de entrada. Se utiliza para representar esos datos de manera compacta y segura.. 8–11

hashing Proceso de realizar un hash. 11

Javascript JavaScript es un lenguaje de programación interpretado, ligero y versátil, utilizado principalmente para crear contenido web interactivo.. 17

Log-In El login (o inicio de sesión) es el proceso de autenticación mediante el cual un usuario se identifica para acceder a un sistema, aplicación, sitio web o dispositivo informático de forma segura. 26

Metamask Metamask es una extensión de navegador que permite a los usuarios interactuar con aplicaciones descentralizadas (DApps) en la red Ethereum, facilitando la gestión de cuentas y la firma de transacciones.. 19, 41

NFT Non Fungible Token, activo digital único que representa propiedad o autenticidad de un objeto, como arte, música o coleccionables almacenado en una blockchain. A diferencia de las criptomonedas 'no fungible' significa que no es divisible. 15, 47, 58

Oráculo Un oráculo en blockchain es un servicio que proporciona datos externos a los contratos inteligentes, permitiéndoles interactuar con información del mundo real, como precios, clima o eventos deportivos.. 58

Pool Grupo de fondos disponibles para prestar en una plataforma DeFi.. 6, 7

Pop-up Ventana emergente que aparece sobre la interfaz de una aplicación o página web para mostrar información o solicitar una acción del usuario, como la autorización de una transacción.. 16

script Conjunto de instrucciones o código escrito en un lenguaje de programación o de scripting que se ejecuta automáticamente para realizar tareas específicas.. 12

SHA-256 Secure Hash Algorithm 256, es una función hash criptográfica que genera un valor único de 256 bits (32 bytes) a partir de una entrada. 11

Smart Contract Contrato inteligente: Script de programación que es capaz de automatizar procesos y transacciones dentro de una red blockchain programable como es la de Ethereum. 3, 68, 69

Smart Contract Programa o código autoejecutable que se ejecuta en una blockchain. Define reglas y condiciones preestablecidas y, cuando se cumplen, ejecuta automáticamente acciones sin necesidad de intermediarios. Se usa en transacciones, acuerdos digitales y aplicaciones descentralizadas.. 4–6, 12–19, 21, 27, 28, 30–33, 40, 46, 57, III

Solidity Lenguaje de programación utilizado para programarSmart Contract. 14, 17

Tipo Address En solidity existe el tipo 'address' utilizado para distinguir que valores corresponden a el de una dirección en la red.. 35, 37

Token Activo digital emitido en una red blockchain que representa un valor, un derecho o una utilidad específica dentro de una plataforma. 14

UI User Interface: Interfaz de usuario. 8

Uniswap Bolsa de intercambio descentralizada basada en Ethereum que automatiza el intercambio de criptomonedas mediante fondos de liquidez, sin intermediarios ni libros de órdenes tradicionales. . 7

Wallet Software o hardware que te permite almacenar, gestionar y firmar transacciones con tus criptomonedas sin depender de un intermediario como un banco.. 15, 16, 59, 60

Web3 Web3 es la evolución de la web que utiliza tecnología blockchain para descentralizar aplicaciones y servicios, permitiendo mayor control, privacidad y propiedad de los datos por parte de los usuarios.. 4, 5, 15, 17

Wei Wei es la unidad más pequeña de Ether (ETH), donde 1 ETH = 1,000,000,000,000,000,000 Wei (10^{18} Wei). Es la unidad de medida utilizada en las transacciones de la blockchain de Ethereum.. 41

Siglas

DNI Documento Naciona de Identidad. 14

EVM Ethereum Virtual Machine, Máquina Virtual de Ethereum. 14–18

OCDE Organización para la Coperación y el Desarrollo Económico. 59

ODS Objetivos de Desarrollo Sostenible. 61, 62

OP Objetivo principal. 2, 44, 47

OS Objetivo secundario. 2, 3, 44, 45, 47

PFG proyecto de fin de grado. 4

PoS Proof of stake, Prueba de participación. 10, 60, 62

PoW Proof of Work, Prueba de trabajo. 10, 60, 62

